UNIT-5

DR. JISHA P

Syllabus

Synchronous sequential circuits: Moore and Mealy FSM, Design and **implementation of sequence detector, serial adder, code converter.**

FPGA based systems: Introduction, basic concepts, Digital design with FPGAs, FPGA based system design.

Synchronous sequential circuits

sequential circuits : Class of circuits in which the outputs depend on the past behavior of the circuit, as well as on the present values of inputs.

If a clock signal is used to control the operation of a sequential circuit; such a circuit is called a synchronous sequential circuit.

Sequential circuit in which no clock signal is used, is called an asynchronous sequential circuit.

Synchronous sequential circuits are realized using combinational logic and one or more flip-flops.

Although the outputs always depend on the present state, they do not necessarily have to depend directly on the primary inputs

Sequential circuits whose outputs depend only on the state of the circuit are of Moore type, while those whose outputs depend on both the state and the primary inputs are of Mealy type.

Sequential circuits are also called finite state machines (FSMs).

Finite State Machines (FSM)

A Finite State Machine (FSM) is a mathematical model used to design sequential logic circuits, where the system can exist in a finite number of states at any given time.

The FSM can change from one state to another in response to some inputs; the change from one state to another is called a **transition**.

An FSM is defined by a list of its states, its initial state, and the inputs that trigger each transition.

FSMs transition between states based on input signals and are widely used in digital design, control systems, communication protocols, and software applications.

FSMs are categorized into two main types:

1. **Deterministic Finite-State Machines (DFSM)**
2. **Non-Deterministic Finite-State Machines (NDFSM)**

Finite State Machines (FSM)

1. Deterministic Finite-State Machine (DFSM)

A **Deterministic FSM** is one in which, for every state and input symbol, there is **exactly one transition** to a next state.

The system's behavior is completely predictable; given the input and current state, the next state is always known.

Suitable for hardware implementations like sequential circuits.

2. Non-Deterministic Finite-State Machine (NDFSM)

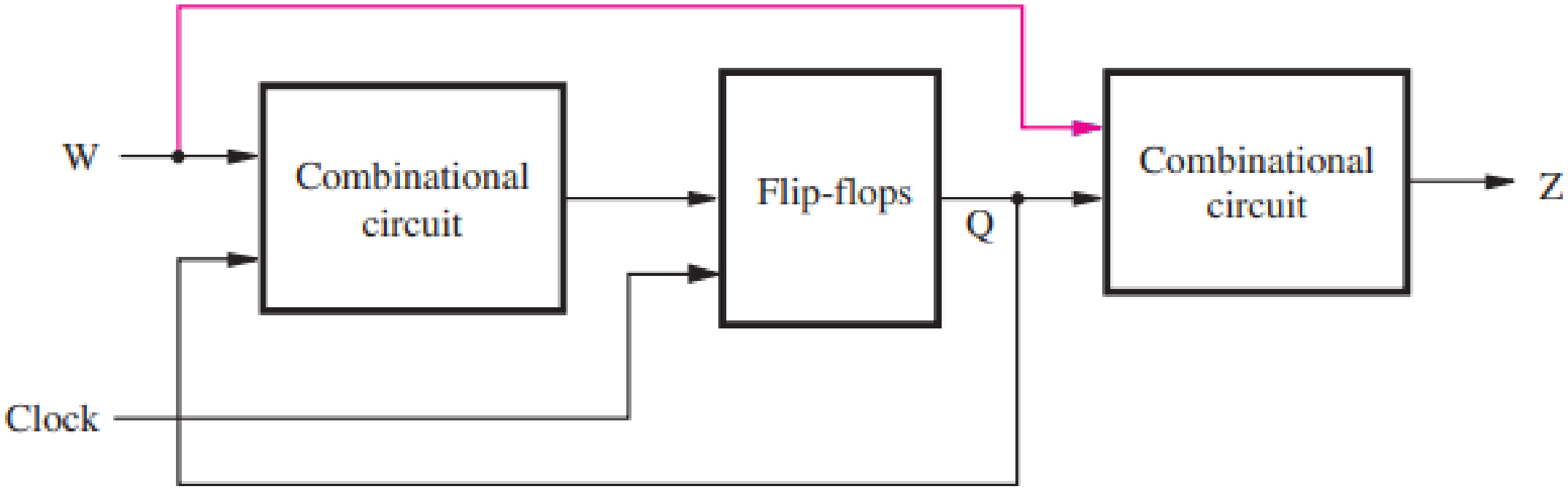
A **Non-Deterministic FSM** allows for **multiple possible transitions** from a given state and input. The next state is not uniquely defined, and the machine can "choose" any one of the available transitions.

The system can take different paths for the same input and state.

Key Differences Between DFSM and NDFSM

Feature	**Deterministic FSM (DFSM)**	**Non-Deterministic FSM (NDFSM)**
Transitions	One transition for each state-input pair	Multiple transitions allowed
Next State	Uniquely determined	Can have multiple possibilities
Complexity	Simpler to design and implement	Conceptually simpler but harder to implement
Practical Implementation	Easy to implement in hardware/software	Needs conversion to DFSM for implementation
Representation of Null Moves	Not applicable	Allows ε-transitions (null input moves)

The general form of a sequential circuit.



Basic Design Steps

Example:

Design a circuit that meets the following specification:

1. The circuit has one input, w, and one output, z.
2. All changes in the circuit occur on the positive edge of a clock signal.
3. The output z is equal to 1 if during two immediately preceding clock cycles the input w was equal to 1. Otherwise, the value of z is equal to 0.

Basic Design Steps

Sequences of input and output signals:

Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w:	0	1	0	1	1	0	1	1	1	0	1
z:	0	0	0	0	0	1	0	0	1	1	0

Input: w

Output: z

Condition: z = 1 if the input w was 1 during the last two consecutive clock cycles.

Timing: All changes occur on the positive edge of the clock.

You need to track the history of w over the last two clock cycles to determine the output z.

State Diagram

1. The first step in designing a finite state machine is to determine how many states are needed and which transitions are possible from one state to another.
2. A good way to begin is to select one particular state as a starting state; this is the state that the circuit should enter when power is first turned on or when a reset signal is applied.
3. For this example let us assume that the starting state is called state A. As long as the input w is 0, the circuit need not do anything, and so each active clock edge should result in the circuit remaining in state A.
4. When w becomes equal to 1, the machine should recognize this, and move to a different state, which we will call state B. This transition takes place on the next active clock edge after w has become equal to 1.

State Diagram

5. In state B, as in state A, the circuit should keep the value of output z at 0, because it has not yet seen w = 1 for two consecutive clock cycles.
6. When in state B, if w is 0 at the next active clock edge, the circuit should move back to state A.
7. However, if w = 1 when in state B, the circuit should change to a third state, called C, and it should then generate an output z = 1. The circuit should remain in state C as long as w = 1 and should continue to maintain z = 1.
8. When w becomes 0, the machine should move back to state A.

Hence three states are needed to implement the desired machine !!

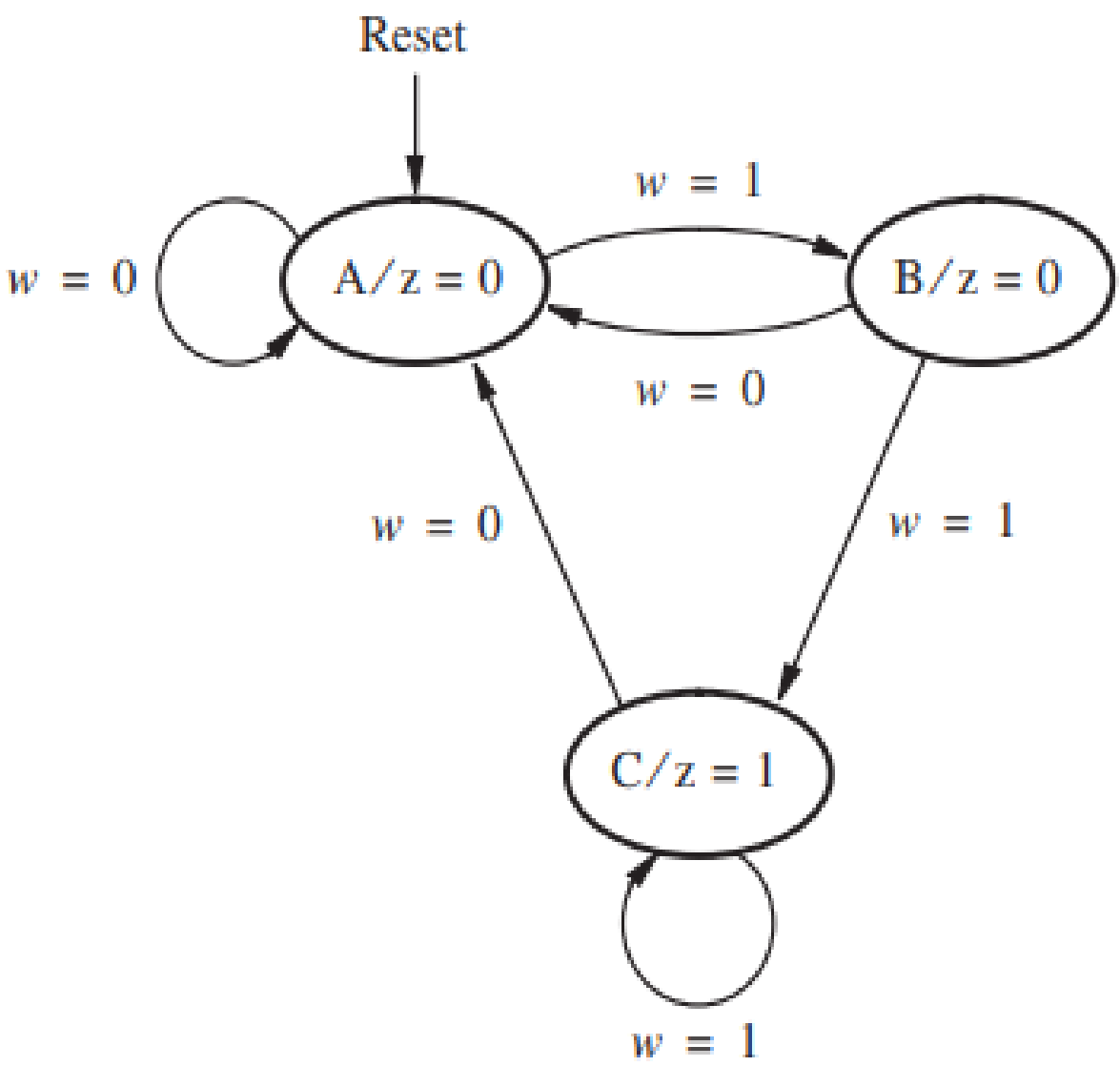
State Diagram

Behaviour of a sequential circuit can be described in several different ways. ‘

The conceptually simplest method is to use a pictorial representation in the form of a state diagram, which is a graph that depicts :

states of the circuit as nodes (circles) and

transitions between states as directed arcs.



State Diagram

States A, B, and C appear as nodes in the diagram. **Node A represents the starting state, and it is also the state that the circuit will reach after an input w = 0 is applied.** In this state the output z should be 0, which is indicated as **A/z=0** in the node. The circuit should remain in state A as long as w = 0, which is indicated by an arc with a label w = 0 that originates and terminates at this node.

The first occurrence of w = 1 (following the condition w = 0) is recorded by moving from state A to state B. This transition is indicated on the graph by an arc originating at A and terminating at B.

The label w = 1 on this arc denotes the input value that causes the transition In state B the output remains at 0, which is indicated as **B/z=0** in the node.

When the circuit is in state B, it will change to state C if w is still equal to 1 at the next active clock edge. In state C the output z becomes equal to 1. If w stays at 1 during subsequent clock cycles, the circuit will remain in state C maintaining z = 1.

However, if w becomes 0 when the circuit is either in state B or in state C, the next active clock edge will cause a transition to state A to take place.

State Table

To proceed with the implementation of the circuit, it is convenient to translate the information contained in the state diagram into a tabular form.

The table indicates all transitions from each present state to the next state for different values of the input signal.

Note that the output z is specified with respect to the present state, namely, the state that the circuit is in at present time.

Present state	Next state		Output z
	$w = 0$	$w = 1$	
A	A	B	0
B	A	C	0
C	A	C	1

State Table

Current State	w = 0	w = 1	Output (z)
A	A	B	0
B	A	C	0
C	A	C	1

State Assignment

- The state table defines the three states in terms of letters A, B, and C.
- When implemented in a logic circuit, each state is represented by a particular valuation (combination of values) of state variables.
- A state variable is a physical representation of a state.
- It is a binary variable (0 or 1) used to encode the states in a digital circuit.
- A combination of state variables uniquely identifies the current state of the FSM.
- Each state variable may be implemented in the form of a flip-flop.

State Assignment

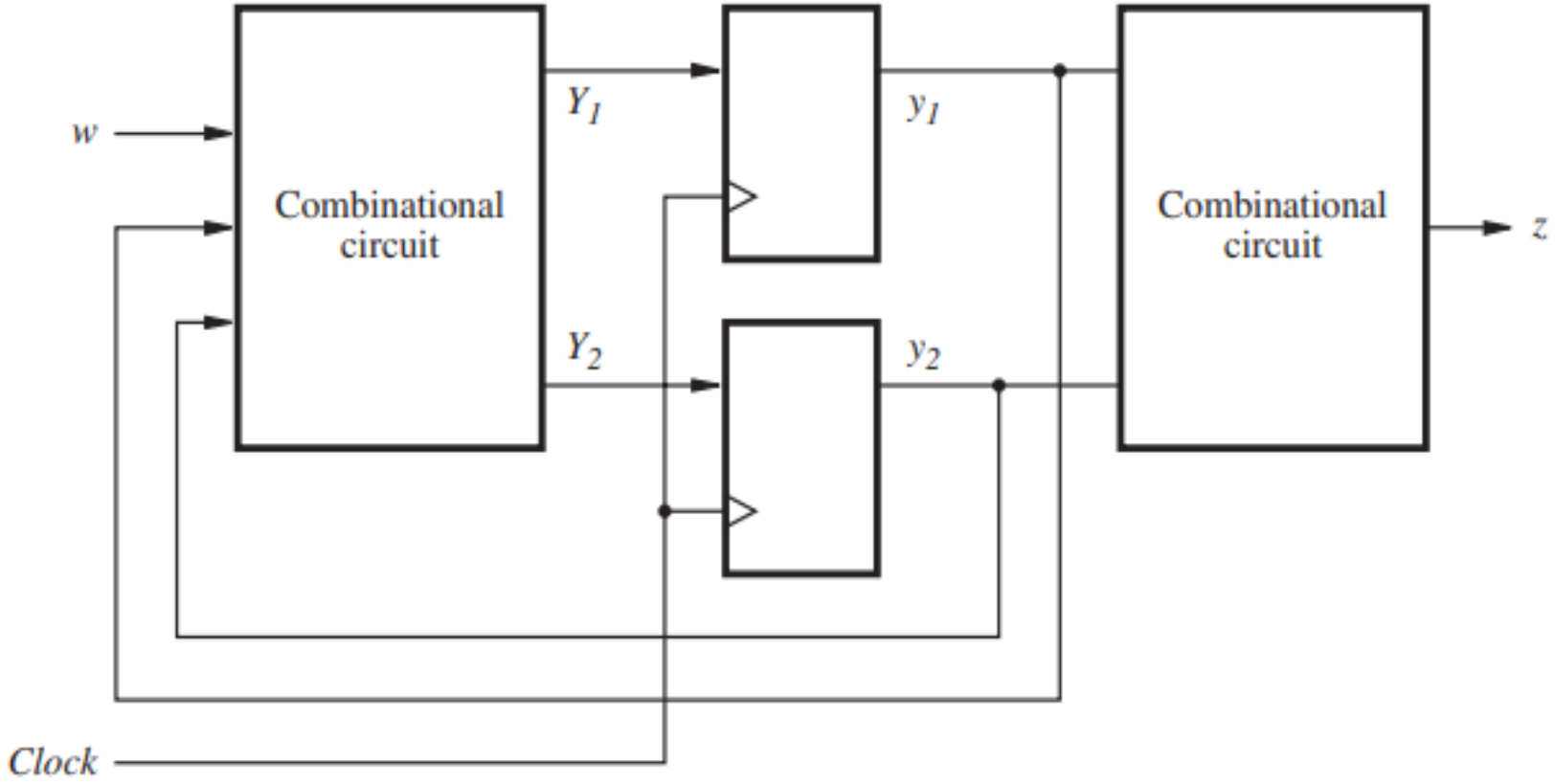
- States are represented using a set of state variables.
- The number of state variables required is determined by the total number of states:
- **Number of State Variables=$\lceil \log_2(\text{Number of States}) \rceil$** ,
- where $\lceil\ \rceil$ denotes rounding up to the nearest integer.
- State variables are implemented using flip-flops (D, T, JK, etc.) in hardware.
- The values of the state variables at a given time determine the current state of the FSM.

State Assignment

Since three states have to be realized, it is sufficient to use two state variables. Let these variables be y_1 and y_2.

Figure indicate the structure of the circuit that implements the required finite state machine.

Two flip-flops represent the state variables.



State Assignment

Thus the block diagram in Figure shows that z is a function of only y1 and y2; our design is of **Moore type.**

We need to design a combinational circuit that uses y1 and y2 as input signals and generates a correct output signal z for all possible valuations of these inputs.

The signals y1 and y2 are also fed back to the combinational circuit that determines the next state of the FSM. This circuit also uses the primary input signal w. Its outputs are two signals, Y1 and Y2, which are used to set the state of the flip-flops. Each active edge of the clock will cause the flip-flops to change their state to the values of Y1 and Y2 at that time.

Therefore, Y1 and Y2 are called the next-state variables, and y1 and y2 are called the present-state variables.

We need to design a combinational circuit with inputs w, y1, and y2, such that for all valuations of these inputs the outputs Y1 and Y2 will cause the machine to move to the next state that satisfies our specification. The next step in the design process is to create a truth table that defines this circuit, as well as the circuit that generates z.

State Assignment

To produce the desired truth table, we assign a specific valuation of variables y1 and y2 to each state.

The states A, B, and C are represented by y_2y_1 = 00, 01, and 10, respectively.

The fourth valuation, y_2y_1 = 11, is not needed in this case.

The type of table is usually called a **state-assigned table**.

This table can serve directly as a truth table for the output z with the inputs y1 and y2.

Although for the next-state functions Y1 and Y2 the table does not have the appearance of a normal truth table, because there are two separate columns in the table for each value of w, it is obvious that the table includes all of the information that defines the next-state functions in terms of valuations of inputs w, y1, and y2.

State-assigned table

Present state	Next state		Output z
	$w = 0$	$w = 1$	
A	A	B	0
B	A	C	0
C	A	C	1

	Present state y_2y_1	Next state $w = 0$ Y_2Y_1	Next state $w = 1$ Y_2Y_1	Output z
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	dd	dd	d

Choice of Flip-Flops and Derivation of Next-State and Output Expressions

From the state-assigned table, we can derive the logic expressions for the next-state and output functions.

The most straightforward choice is to use D-type flip-flops, because in this case the values of Y1 and Y2 are simply clocked into the flip-flops to become the new values of y1 and y2.

Karnaugh maps method is used to verify the validity of the expressions.

The fourth valuation, $y_2y_1 = 11$, should never occur in the circuit because the circuit is constrained to move only within states A, B, and C; therefore, we can choose to treat this valuation as a don’t-care condition.

$w \backslash y_2y_1$	00	01	11	10
0	0	0	d	0
1	1	0	d	0

	Ignoring don't cares	Using don't cares
Y_1	$w\bar{y}_1\bar{y}_2$	$w\bar{y}_1\bar{y}_2$

$w \backslash y_2y_1$	00	01	11	10
0	0	0	d	0
1	0	1	d	1

	Ignoring don't cares	Using don't cares
Y_2	$wy_1\bar{y}_2 + w\bar{y}_1y_2$	$Y_2 = wy_1 + wy_2$ $= w(y_1 + y_2)$

$y_2 \backslash y_1$	0	1
0	0	0
1	1	d

	Ignoring don't cares	Using don't cares
z	$\bar{y}_1y_2$	$z = y_2$

Choice of Flip-Flops and Derivation of Next-State and Output Expressions

Using the don't cares to simplify the expressions, we obtain,

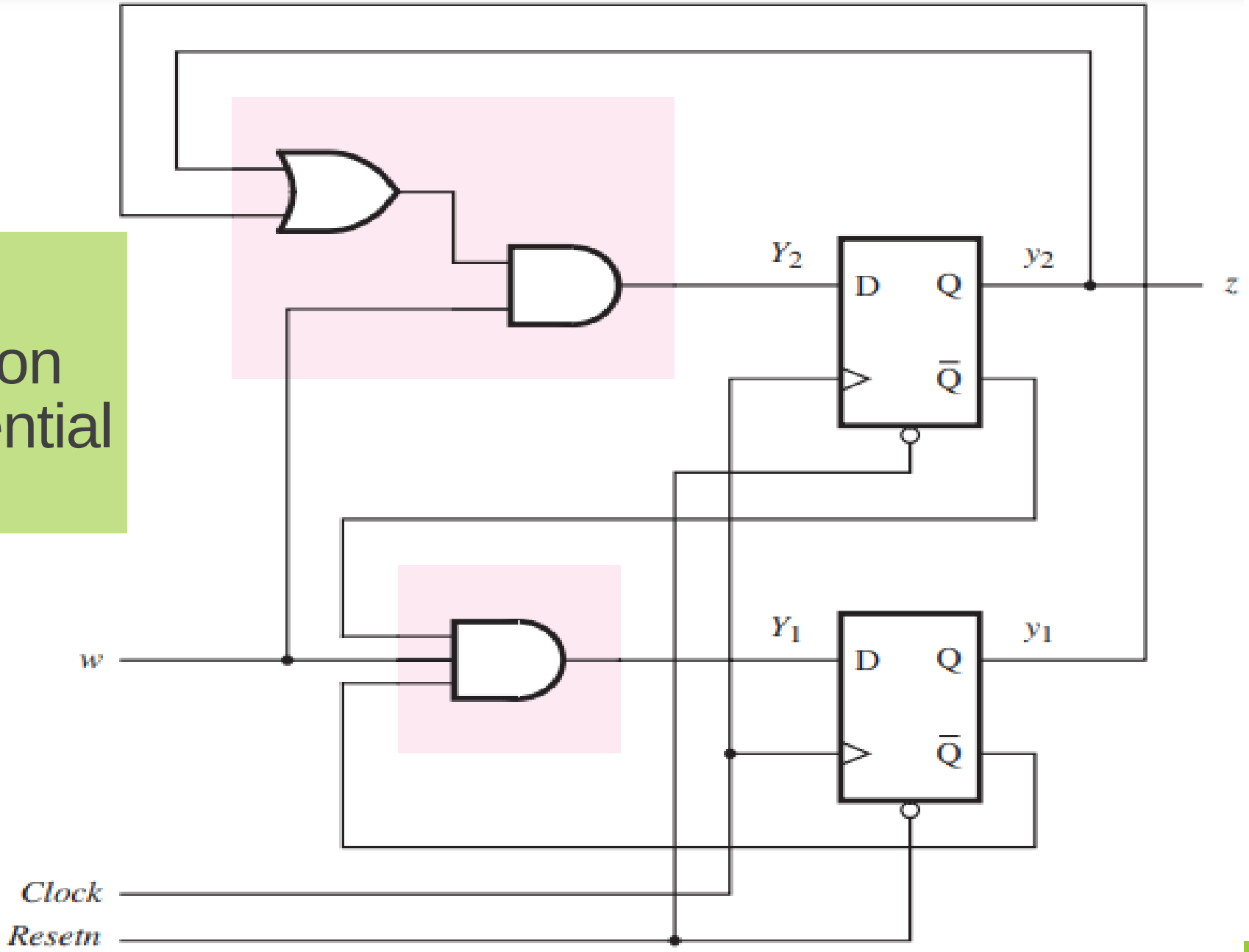
$$Y_1 = w\bar{y}_1\bar{y}_2$$

$$Y_2 = w(y_1 + y_2)$$

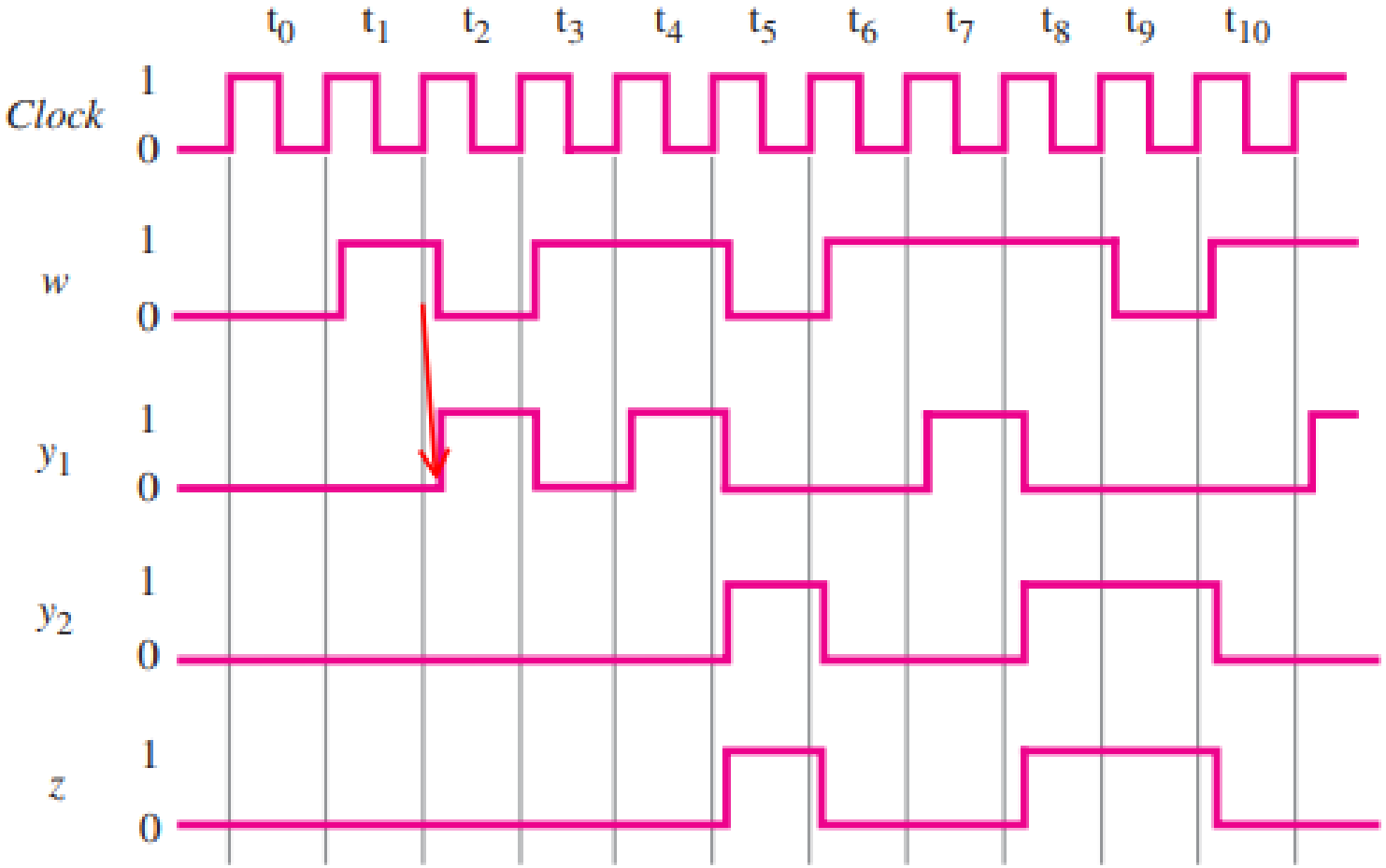
$$z = y_2$$

If we do not use don't cares, then the resulting expressions are slightly more complex;
SinceD1 = Y1 and D2 = Y2, the logic circuit that corresponds to the preceding expressions is implemented as shown in Figure.
In this a clock signal is included, and the circuit is provided with an active-low reset capability. Connecting the clear input on the flip-flops to an external Resetn signal, as shown in the figure, provides a simple means for forcing the circuit into a known state.
If we apply the signal Resetn = 0 to the circuit, then both flip-flops will be cleared to 0, placing the FSM into the state y_2y_1 = 00.

Final implementation of the sequential circuit



Timing Diagram



Timing Diagram

The amount of delay from the clock edge depends on the propagation delays through the flip-flops.

Note that the input signal w is also shown to change slightly after the active edge of the clock.

This is a good assumption because in a typical digital system an input such as w would be just an output of another circuit that is synchronized by the same clock.

A key point to observe is that even though w changes slightly after the active clock edge, and thus the value of w is equal to 1 (or 0) for almost the entire clock cycle, no change in the circuit will occur until the beginning of the next

Thus the value of w must be equal to 1 for two clock cycles if the circuit is to reach state C and generate the output z = 1.

Summary of the steps involved in designing a synchronous sequential circuit as follows:

1. Obtain the specification of the desired circuit.

2. Derive the states for the machine by first selecting a starting state. Then, given the specification of the circuit, consider all valuations of the inputs to the circuit and create new states as needed for the machine to respond to these inputs. To keep track of the states as they are visited, create a state diagram. When completed, the state diagram shows all states in the machine and gives the conditions under which the circuit moves from one state to another.

3. Create a state table from the state diagram. Alternatively, it may be convenient to directly create the state table in step 2, rather than first creating a state diagram.

Summary of the steps involved in designing a synchronous sequential circuit as follows:

4. In our sequential circuit example, there were only three states; hence it was a simple matter to create the state table that does not contain more states than necessary. However, in practice it is common to deal with circuits that have a large number of states. In such cases it is unlikely that the first attempt at deriving a state table will produce optimal results. Almost certainly we will have more states than is really necessary. This can be corrected by a procedure that minimizes the number of states.

5. Decide on the number of state variables needed to represent all states and perform the state assignment. There are many different state assignments possible for a given sequential circuit. Some assignments may be better than others. In the preceding example we used what seemed to be a natural state assignment.

Summary of the steps involved in designing a synchronous sequential circuit as follows:

6. Choose the type of flip-flops to be used in the circuit. Derive the next-state logic expressions to control the inputs to all flip-flops and then derive logic expressions for the outputs of the circuit.

7. Implement the circuit as indicated by the logic expressions.

Example- 2

Design a control mechanism that swaps the contents of registers R1 and R2.

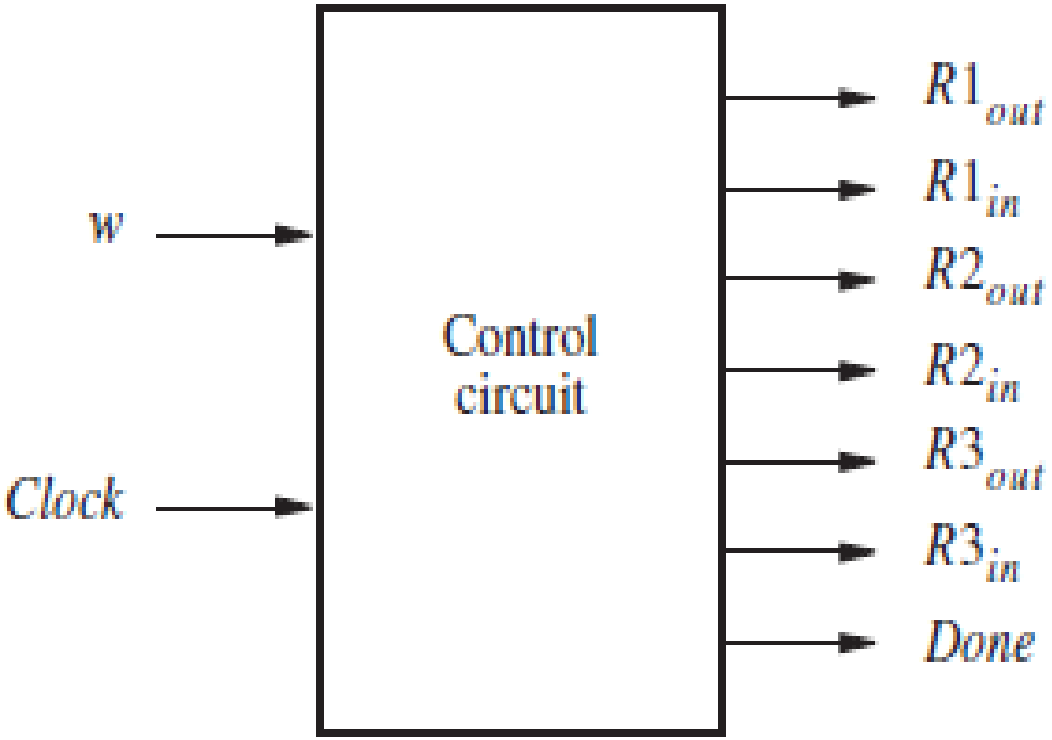
We will now design the desired control mechanism, using the finite state machine approach.

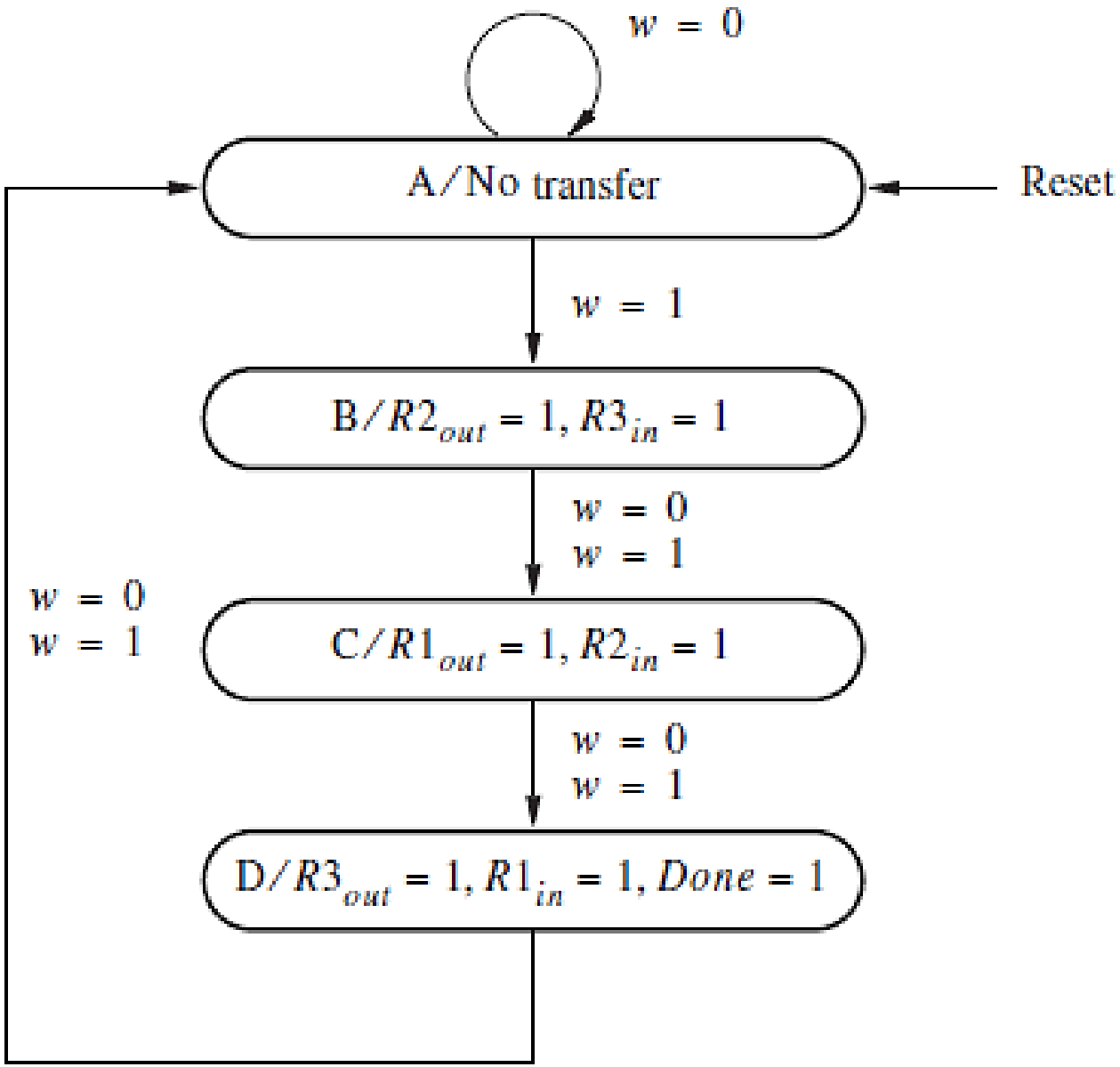
The contents of registers R1 and R2 can be swapped using register R3 as a temporary storage location as follows:

1. The contents of R2 are first loaded into R3, using the control signals R2out = 1 and R3in = 1.
2. Then the contents of R1 are transferred into R2, using R1out = 1 and R2in = 1.
3. Finally, the contents of R3 (which are the previous contents of R2) are transferred into R1, using R3out = 1 and R1in = 1.
4. Since this step completes the required swap, we will indicate that the task is completed by setting the signal Done = 1.

Assume that the swapping is performed in response to a pulse on an input signal called w, which has a duration of one clock cycle.

State diagram





State diagram

In the starting state, A, no transfer is indicated, and all output signals are 0. The circuit remains in this state until a request to swap arrives in the form of w changing to 1.

In state B the signals required to transfer the contents of R2 into R3 are asserted. The next active clock edge places these contents into R3. It also causes the circuit to change to state C, regardless of whether w is equal to 0 or 1.

In this state the signals for transferring R1 into R2 are asserted. The transfer takes place at the next active clock edge, and the circuit changes to state D regardless of the value of w.

The final transfer, from R3 to R1, is performed on the clock edge that leaves state D, which also causes the circuit to return to state A.

State Table

Present state	Next state		Outputs						
	$w = 0$	$w = 1$	$R1_{out}$	$R1_{in}$	$R2_{out}$	$R2_{in}$	$R3_{out}$	$R3_{in}$	$Done$
A	A	B	0	0	0	0	0	0	0
B	C	C	0	0	1	0	0	1	0
C	D	D	1	0	0	1	0	0	0
D	A	A	0	1	0	0	1	0	1

Since there are four states, it is necessary to use two state variables, *y*2 and *y*1. A straight forward state assignment where the states *A*, *B*, *C*, and *D* are assigned the valuations *y*2*y*1 = 00, 01, 10, and 11, respectively.

State - Assigned Table

	Present state	Next state		Outputs						
		$w = 0$	$w = 1$							
	y_2y_1	Y_2Y_1	Y_2Y_1	$R1_{out}$	$R1_{in}$	$R2_{out}$	$R2_{in}$	$R3_{out}$	$R3_{in}$	$Done$
A	0 0	0 0	0 1	0	0	0	0	0	0	0
B	0 1	1 0	1 0	0	0	1	0	0	1	0
C	1 0	1 1	1 1	1	0	0	1	0	0	0
D	1 1	0 0	0 0	0	1	0	0	1	0	1

State - Assigned Table

Using this assignment and D-type flip-flops, the next-state expressions can be derived as shown in Figure.

Derivation of next-state expressions for the sequential circuit:

$w \backslash y_2y_1$	00	01	11	10
0				1
1	1			1

$$Y_1 = w\bar{y}_1 + \bar{y}_1 y_2$$

$w \backslash y_2y_1$	00	01	11	10
0		1		1
1		1		1

$$Y_2 = y_1\bar{y}_2 + \bar{y}_1 y_2$$

Next - State expressions

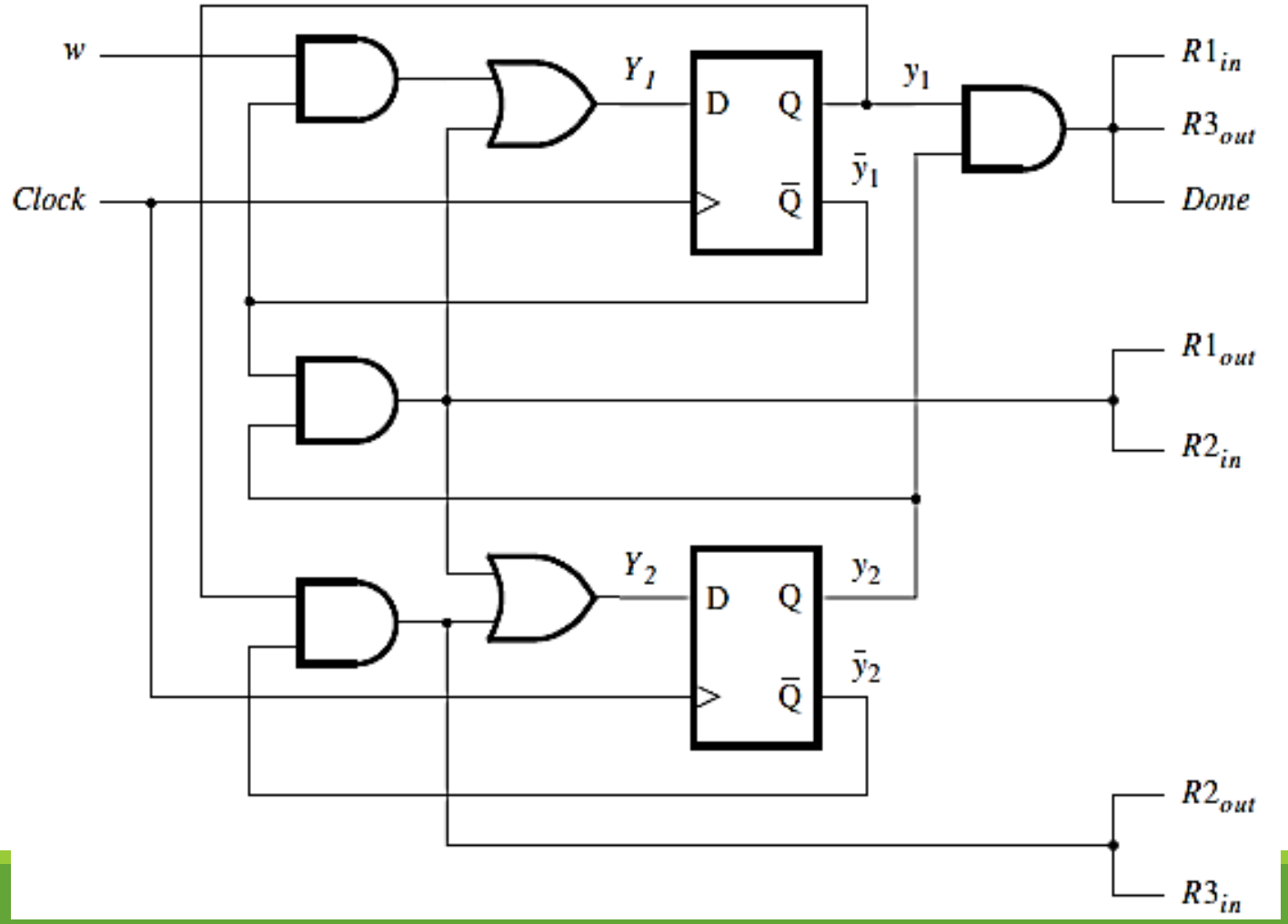
The next-state expressions are:

$$Y_1 = w\bar{y}_1 + \bar{y}_1 y_2$$
$$Y_2 = y_1\bar{y}_2 + \bar{y}_1 y_2$$

The output control signals are derived as:

$$R1_{out} = R2_{in} = \bar{y}_1 y_2$$
$$R1_{in} = R3_{out} = Done = y_1 y_2$$
$$R2_{out} = R3_{in} = y_1\bar{y}_2$$

Final implementation of the sequential circuit



State-Assignment Problem

	Present state y_2y_1	Next state $w=0$ Y_2Y_1	Next state $w=1$ Y_2Y_1	Output z
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	dd	dd	d

	Present state y_2y_1	Next state $w=0$ Y_2Y_1	Next state $w=1$ Y_2Y_1	Output z
A	00	00	01	0
B	01	00	11	0
C	11	00	11	1
	10	dd	dd	d

State-Assignment Problem

The next-state and output expressions derived from the figure will be:

$$Y_1 = D_1 = w$$

$$Y_2 = D_2 = wy_1$$

$$z = y_2$$

Previous case equations

$$Y_1 = w\bar{y}_1\bar{y}_2$$

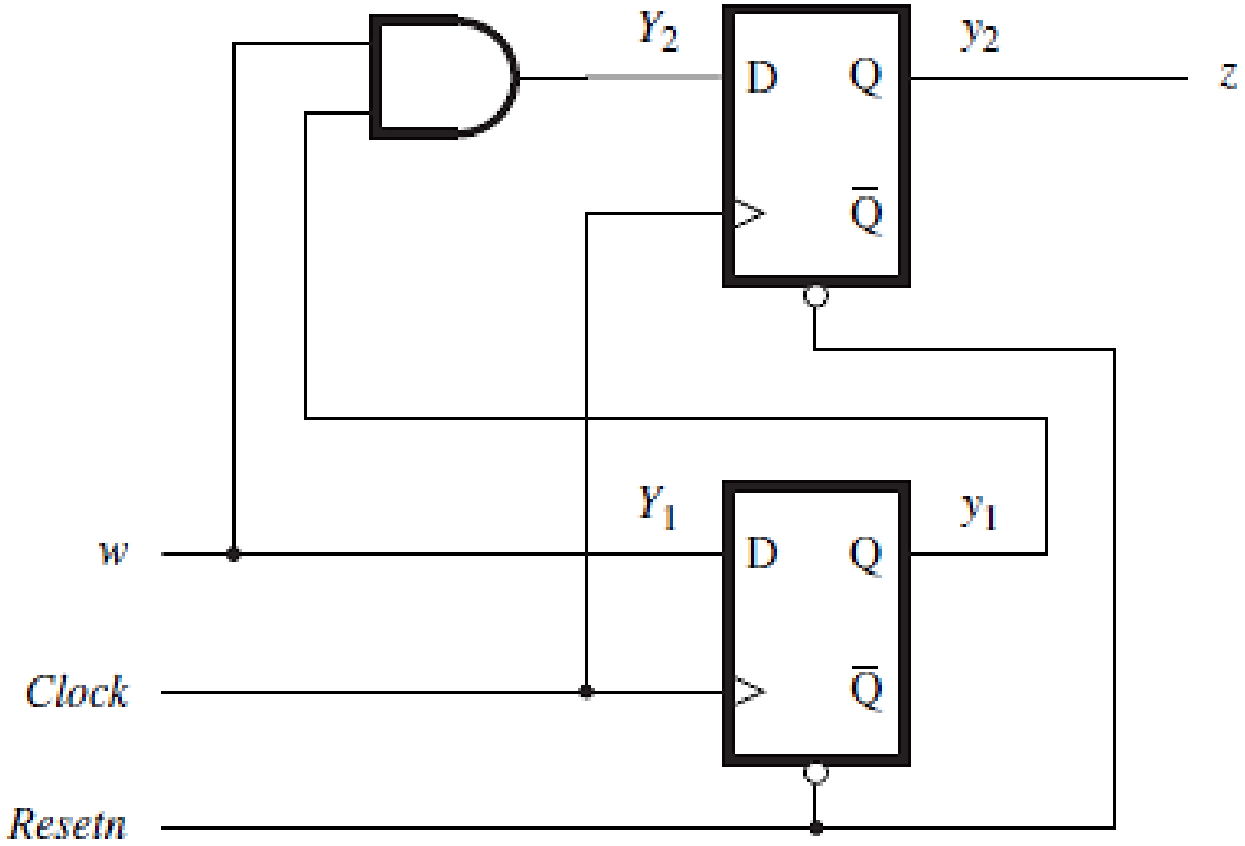
$$Y_2 = w(y_1 + y_2)$$

$$z = y_2$$

State-Assignment Problem

These expressions define the circuit shown in Figure.

Comparing this circuit with the previous one, we see that the cost of the new circuit is lower because it requires fewer gates.



State-Assignment Problem

In general, circuits are much larger than our example, and different state assignments can have a substantial effect on the cost of the final implementation.

While highly desirable, it is often impossible to find the best state assignment for a large circuit.

The exhaustive approach of trying all possible state assignments is not practical because the number of available state assignments is huge.

CAD tools usually perform the state assignment using heuristic techniques. These techniques are usually proprietary, and their details are seldom published.

One-Hot Encoding

Another interesting possibility is to use as many state variables as there are states in a sequential circuit.

In this method, for each state all but one of the state variables are equal to 0.

The variable whose value is 1 is deemed to be “hot.”

The approach is known as the one-hot encoding method.

One-Hot Encoding

Consider state table of Example -1:

Because there are three states, it is necessary to use three state variables.

The chosen assignment is to represent the states A, B, and C using the valuations

$y_3y_2y_1$ = 001, 010, and 100, respectively.

The remaining five valuations of the state variables are not used.

They can be treated as don't cares in the derivation of the next-state and output expressions.

Present state	Next state		Output z
	$w = 0$	$w = 1$	
A	A	B	0
B	A	C	0
C	A	C	1

	Present state	Next state		Output
		$w = 0$	$w = 1$	z
	$y_3y_2y_1$	$Y_3Y_2Y_1$	$Y_3Y_2Y_1$	
A	0 0 1	0 0 1	0 1 0	0
B	0 1 0	0 0 1	1 0 0	0
C	1 0 0	0 0 1	1 0 0	1

Present state	Next state $w = 0$	Next state $w = 1$	Output z
A	A	B	0
B	A	C	0
C	A	C	1

	Present state y_2y_1	Next state $w = 0$ Y_2Y_1	Next state $w = 1$ Y_2Y_1	Output z
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	*dd*	*dd*	*d*

	Present state $y_3y_2y_1$	Next state $w = 0$ $Y_3Y_2Y_1$	Next state $w = 1$ $Y_3Y_2Y_1$	Output z
A	0 0 1	0 0 1	0 1 0	0
B	0 1 0	0 0 1	1 0 0	0
C	1 0 0	0 0 1	1 0 0	1

One-Hot Encoding

Using this assignment, the resulting expressions are:

$$Y_1 = \overline{w}$$

$$Y_2 = wy_1$$

$$Y_3 = w\overline{y}_1$$

$$z = y_3$$

Common FSM encoding options are:

- One-hot code
- Binary code
- Gray code
- Random code

These expressions are not simpler than those obtained in the previous case state assignment. Although in this case the one-hot assignment is not advantageous, there are many cases where this approach is attractive.

One-Hot Encoding

	Present state	Next state		Outputs						
		$w = 0$	$w = 1$							
	$y_4y_3y_2y_1$	$Y_4Y_3Y_2Y_1$	$Y_4Y_3Y_2Y_1$	$R1_{out}$	$R1_{in}$	$R2_{out}$	$R2_{in}$	$R3_{out}$	$R3_{in}$	$Done$
A	0001	0001	0010	0	0	0	0	0	0	0
B	0010	0100	0100	0	0	1	0	0	1	0
C	0100	1000	1000	1	0	0	1	0	0	0
D	1000	0001	0001	0	1	0	0	1	0	1

$$Y_1 = \overline{w}y_1 + y_4$$
$$Y_2 = wy_1$$
$$Y_3 = y_2$$
$$Y_4 = y_3$$

These expressions are simpler than those derived in Example-2, but four flip-flops are needed, rather than two!!

$$R1_{out} = R2_{in} = y_3$$
$$R1_{in} = R3_{out} = Done = y_4$$
$$R2_{out} = R3_{in} = y_2$$

Here, $\mathbf{y_4y_3y_2y_1}$ = 0001, 0010, 0100, and 1000, respectively.
Treating the remaining 12 valuations of the state variables as don't cares, the next-state expressions are:

Previous case equations are:

$$Y_1 = w\overline{y}_1 + \overline{y}_1y_2$$
$$Y_2 = y_1\overline{y}_2 + \overline{y}_1y_2$$

$$R1_{out} = R2_{in} = \overline{y}_1y_2$$
$$R1_{in} = R3_{out} = Done = y_1y_2$$
$$R2_{out} = R3_{in} = y_1\overline{y}_2$$

One-Hot Encoding

- An important feature of the one-hot state assignment is that it often leads to simpler output expressions than do assignments with the minimal number of state variables.
- Simpler output expressions may lead to a faster circuit.
- For instance, if the outputs of the sequential circuit are just the outputs of the flip-flops, as is the case in our example, then these output signals are valid as soon as the flip-flops change their states.
- If more complex output expressions are involved, then the propagation delay through the gates that implement these expressions must be taken into account.

Mealy State Model

- State transition machines that produce output are classified into Moore Machine and Mealy Machine.
- **Moore machine**

  The output of a Moore machine is only determined by the current state of the machine, and not by the current input. This means that the output only changes when the machine moves to a new state.

- **Mealy machine**

  The output of a Mealy machine is determined by both the current state and the current input. This means that the output can change whenever the input changes, even if the machine remains in the same state.

Example- 3

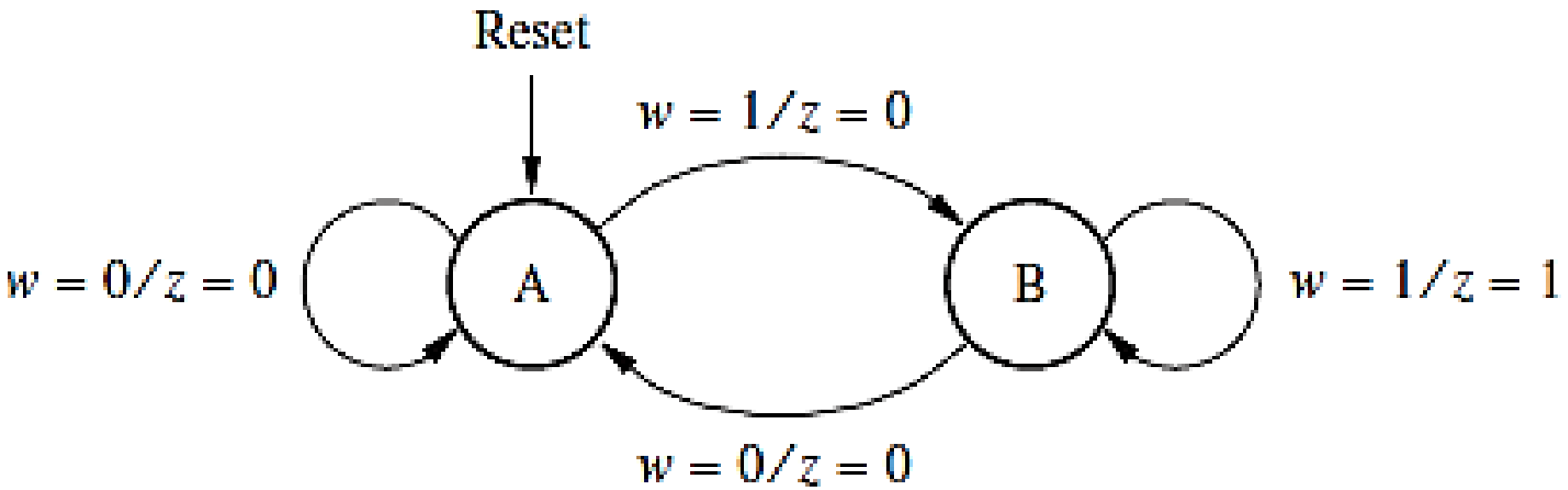
Suppose in Example-1, output z should be equal to 1 in the same clock cycle when the second occurrence of w = 1 is detected.

Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w:	0	1	0	1	1	0	1	1	1	0	1
z:	0	0	0	0	1	0	0	1	1	0	0

Example- 3

- To see how we can realize the behavior given in this table, we begin by selecting a starting state, A.
- As long as w = 0, the machine should remain in state A, producing an output z = 0.
- When w = 1, the machine has to move to a new state, B, to record the fact that an input of 1 has occurred.
- If w remains equal to 1 when the machine is in state B, which happens if w = 1 for at least two consecutive clock cycles, the machine should remain in state B and produce an output z = 1.
- As soon as w becomes 0,z should immediately become 0 and the machine should move back to state A at the next active edge of the clock.

Example- 3 : State diagram of an FSM



Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w:	0	1	0	1	1	0	1	1	1	0	1
z:	0	0	0	0	1	0	0	1	1	0	0

Example- 3 : State diagram of an FSM

- The diagram indicates that if the machine is in state A, it will remain in state A if w = 0 and the output will be 0.
- This is indicated by an arc with the label w = 0/z = 0. When w becomes 1, the output stays at 0 until the machine moves to state B at the next active clock edge. This is denoted by the arc from A to B with the label w = 1/z = 0.
- In state B the output will be 1 if w = 1, and the machine will remain in state B, as indicated by the label w=1/z=1 on the corresponding arc.
- However, if w = 0 in state B, then the output will be 0 and a transition to state A will take place at the next active clock edge.
- A key point to understand is that during the present clock cycle the output value corresponds to the label on the arc emanating from the present-state node.

Example- 3 :State table & State-assigned table

Present state	Next state		Output z	
	$w = 0$	$w = 1$	$w = 0$	$w = 1$
A	A	B	0	0
B	A	B	0	1

The table shows that the output *z* depends on the present value of input *w* and not just on the present state.

	Present state	Next state		Output	
		$w = 0$	$w = 1$	$w = 0$	$w = 1$
	y	Y	Y	z	z
A	0	0	1	0	0
B	1	0	1	0	1

Because there are only two states, it is sufficient to use a single state variable, *y*.

Example- 3 : Logical expressions

The next state Y depends on the present state y and input w.

From the table:

- When y=0 and w=0, Y=0.
- When y=0 and w=1, Y=1.
- When y=1 and w=0, Y=0.
- When y=1 and w=1, Y=1.

So,

Y = w

The output z depends on the present state y and input w.

From the table:

When y=0 and w=0, z=0.

When y=0 and w=1, z=0.

When y=1 and w=0, z=0.

When y=1 and w=1, z=1

Observing this, z becomes 1 only when both y=1 and w=1. T his corresponds to the AND operation between y and w.

Z = w .y

Example- 3: Circuit

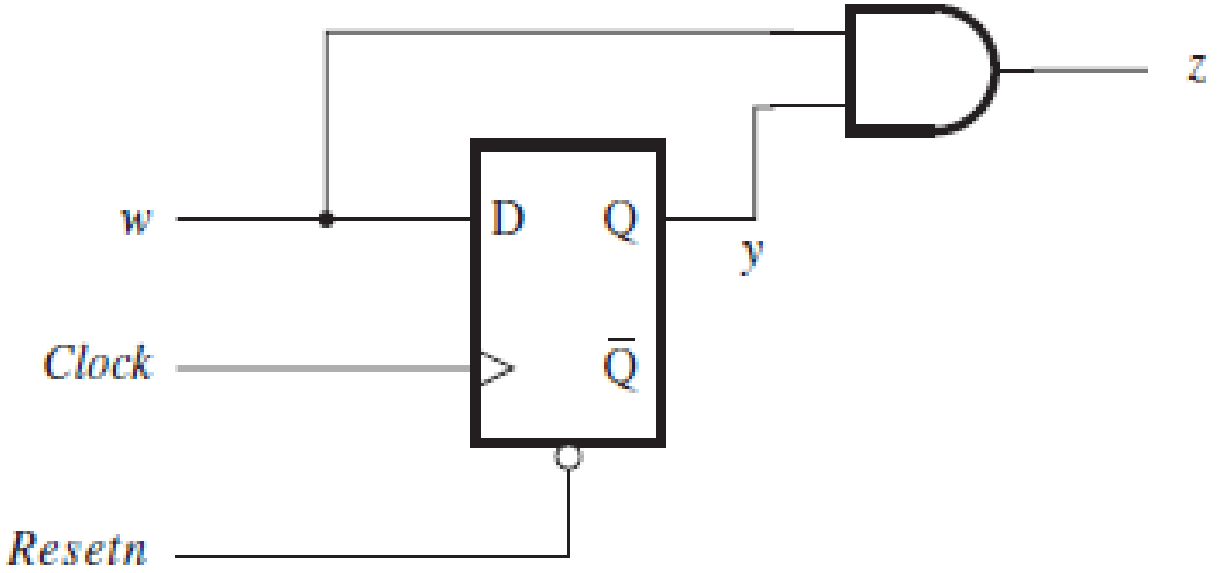
Y only becomes 1 when w=1, irrespective of y. Thus, Y is directly dependent on w.

Assuming that y is realized as a D-type flip-flop, the required next-state and output expressions are:

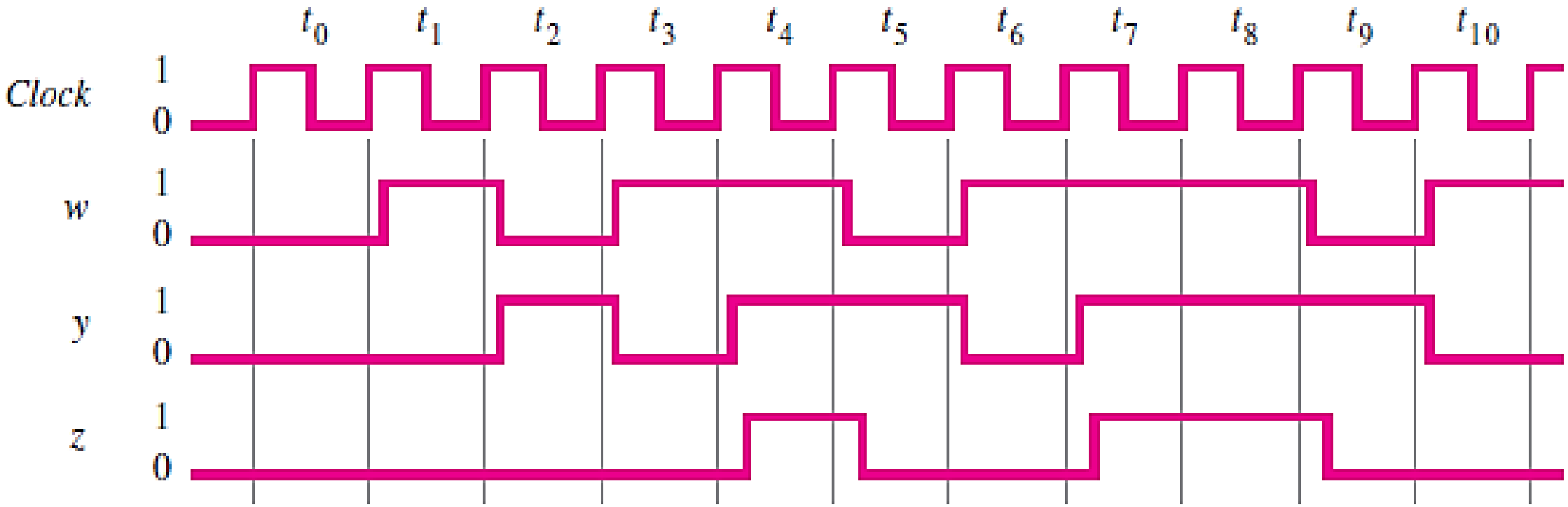
$$Y = D = w$$

$$z = wy$$

Circuit:



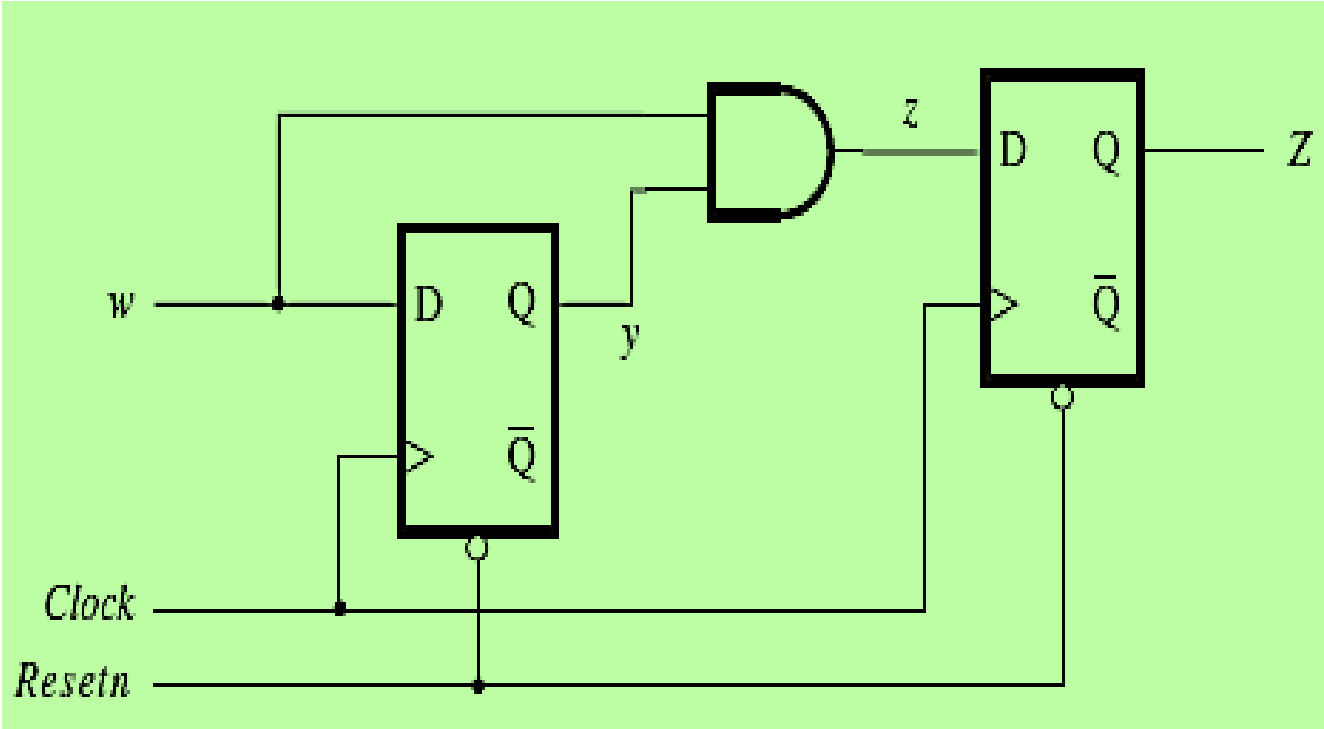
Example- 3: Timing diagram



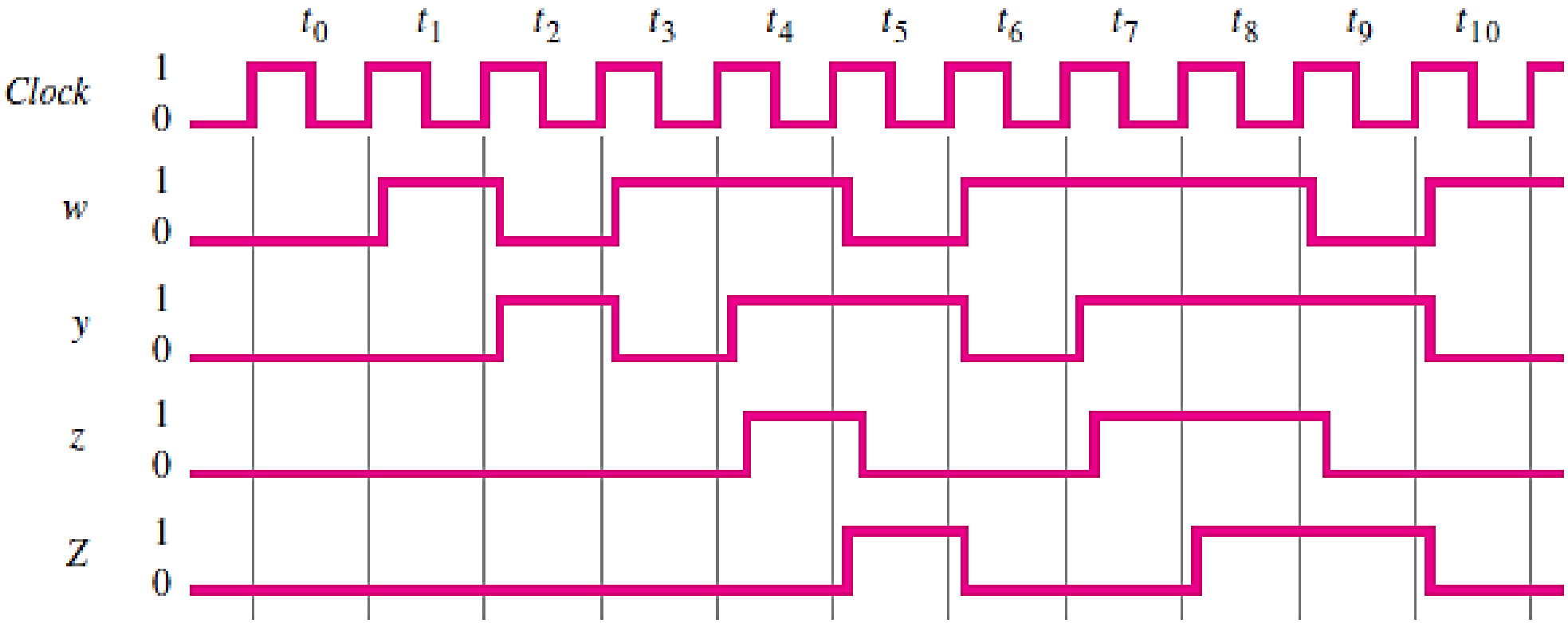
Example- 3: Mealy-type circuit to Moore-type circuit

If we wanted to produce exactly the same output behavior needed in Example 1, using the Mealy approach, we could modify the circuit obtained by adding another flip-flop as shown in Figure.

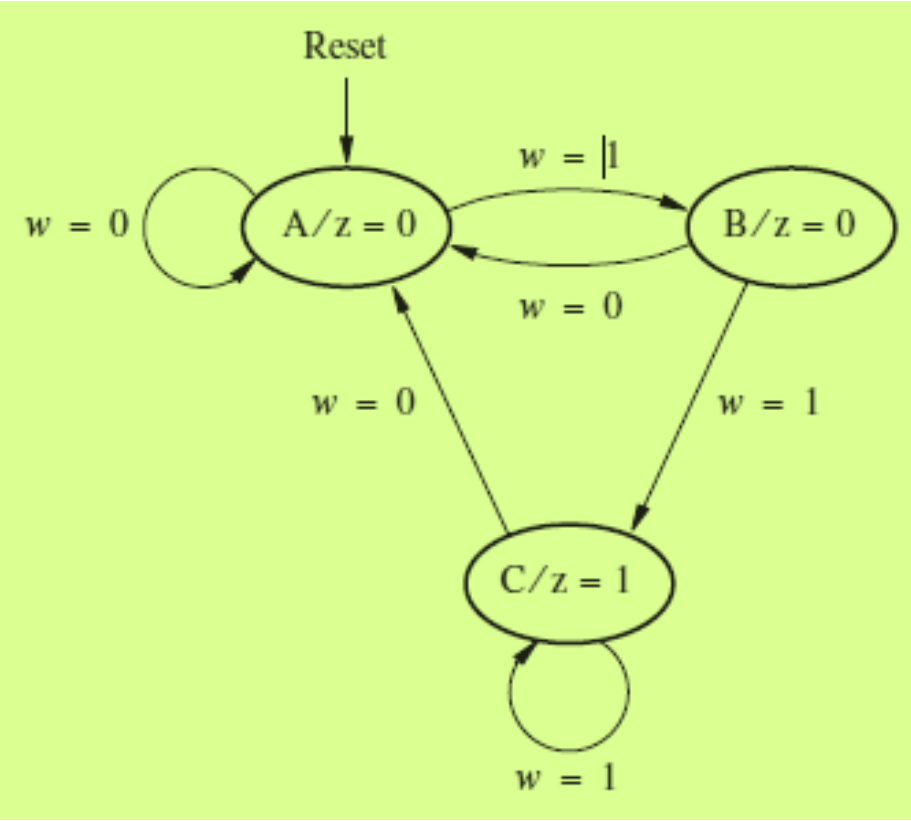
This flip-flop merely delays the output signal, Z, by one clock cycle with respect to z, as indicated in the timing diagram. By making this change, we effectively turn the Mealy-type circuit into a Moore-type circuit with output Z.

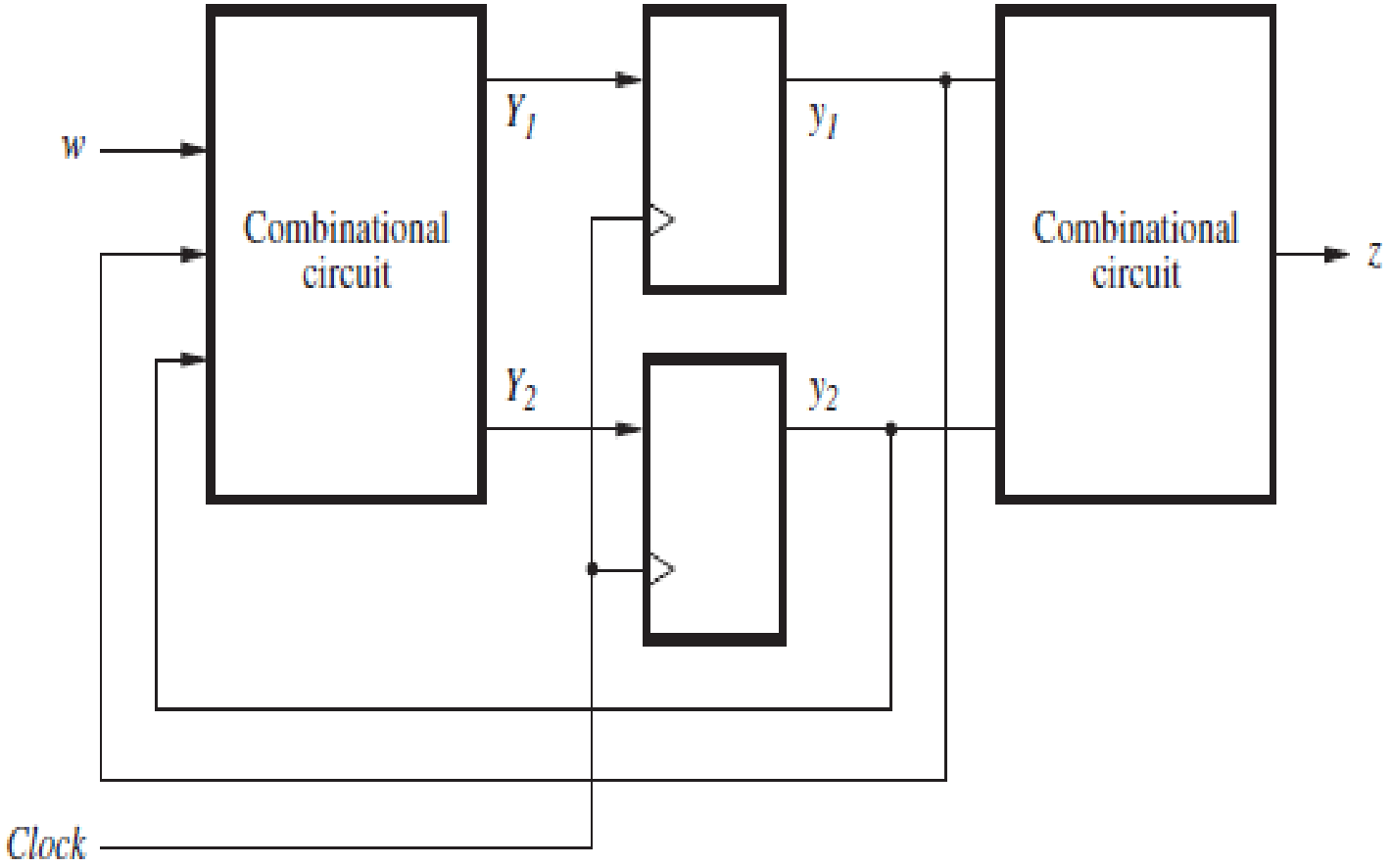


Example- 3: Modified Timing diagram



Verilog Code for Moore-Type FSMs





Verilog Code for Moore-Type FSMs

```
module simple (Clock, Resetn, w, z);
input Clock, Resetn, w;    output z;
reg [2:1] y, Y;
parameter [2:1] A = 2'b00, B = 2'b01, C = 2'b10;
// Define the next state combinational circuit
always @(w or y)
case (y)
A: if (w) Y = B;
else Y = A;
B: if (w) Y = C;
else Y = A;
C: if (w) Y = C;
else Y = A;
default: Y = 2'bxx;
endcase
```

```
// Define the sequential block
always @(negedge Resetn or posedge Clock)
if (Resetn == 0) y <= A;
else y <= Y;
// Define output
assign z = (y == C);
endmodule
```

	Present state	Next state		Output
		$w = 0$	$w = 1$	z
	y_2y_1	Y_2Y_1	Y_2Y_1	
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	dd	dd	d

Verilog Code for Moore-Type FSMs

The module simple has inputs Clock, Resetn, and w, and output z.

Two-bit vectors y and Y represent the present and the next state of the machine, respectively.

The state values are assigned in the parameter statement.

The state transitions are specified by two separate always blocks.

The first block describes the required combinational circuit. The values of the next state vector Y change in response to the changes in w and y signals, which are given in the sensitivity list. The circuit is defined in a case statement to give the value of Y for each value of y and w. Each case alternative corresponds to a present state of the machine, and the associated if-else statement specifies the next state to be reached. Since there are only three states specified as possible alternatives, we have included a default clause which indicates to the Verilog compiler that the unassigned fourth state can be treated as a don’t care condition. This portion of the code corresponds to the combinational circuit on the left side of the circuit. The second always block introduces flip-flops into the circuit. Its sensitivity list comprises the reset and clock signals. Asynchronous reset is performed when the Resetn input goes to 0, placing the FSM into state A. The else clause stipulates that after each positive clock edge the y signal should take the value of the Y signal, thus implementing the state changes. The else clause stipulates that after each positive clock edge the y signal should take the value of the Y signal, thus implementing the state changes.

Example:4 - Serial adder

Design a circuit that will perform serial addition, dealing with a pair of bits in one clock cycle:

The process starts by adding bits *a*0 and *b*0.

In the next clock cycle, bits *a*1 and *b*1 are added, including a possible carry from the bit-position 0, and so on.

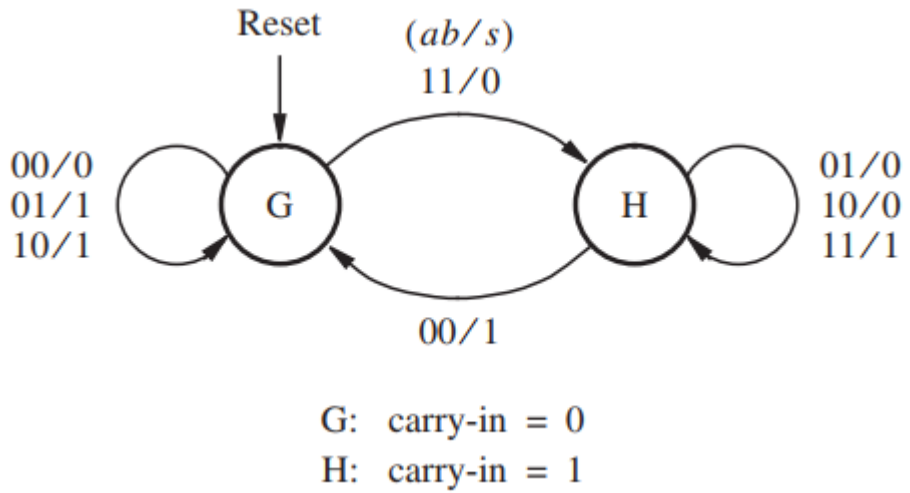
This cannot be a combinational circuit because different actions will have to be taken, depending on the value of the carry from the previous bit position.

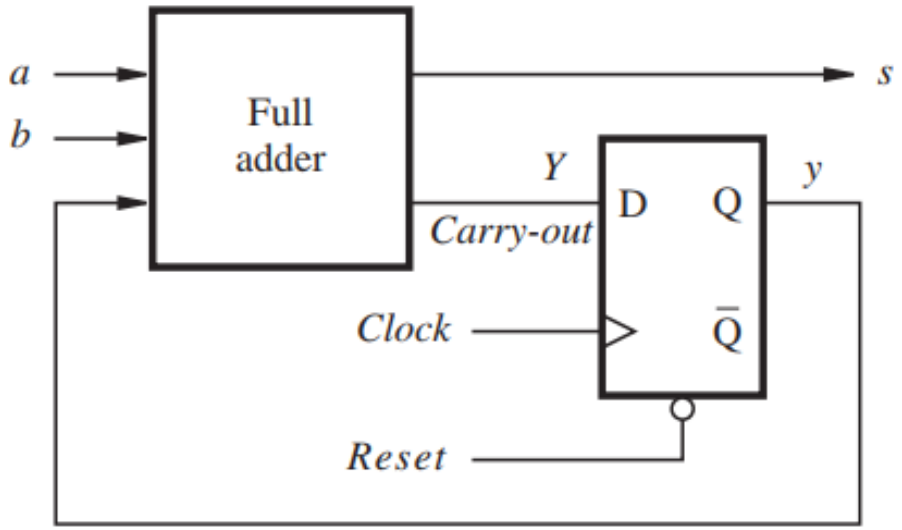
$$A = a_{n-1}a_{n-2}\cdots a_0$$

$$B = b_{n-1}b_{n-2}\cdots b_0$$

$$Sum = s_{n-1}s_{n-2}\cdots s_0.$$

Serial adder - Mealy





Let G and H denote the states where the carry-in-values are 0 and 1.
Output value ***s*** depends on both the state and the present value of inputs ***a*** and ***b***.

Serial adder - Mealy

Input valuation	Output (s)	State
00	0	FSM will remain in same state G
01,10	1	FSM will remain in same state G
11	0	FSM moves to state H
01,10	0	FSM will remain in same state H
11	1	FSM will remain in same state H
00	1	FSM moves to state G

The next state and output equations are:

$$Y = ab + ay + by$$

$$s = a \oplus b \oplus y$$

Serial adder - Mealy

Present state	Next state				Output s			
	$ab = 00$	01	10	11	00	01	10	11
G	G	G	G	H	0	1	1	0
H	G	H	H	H	1	0	0	1

Fig: State table for the Mealy type serial adder FSM

Present state	Next state				Output			
	$ab = 00$	01	10	11	00	01	10	11
y	Y				s			
0	0	0	0	1	0	1	1	0
1	0	1	1	1	1	0	0	1

Fig: State-assigned table for the Mealy type serial adder FSM

Serial adder - Mealy

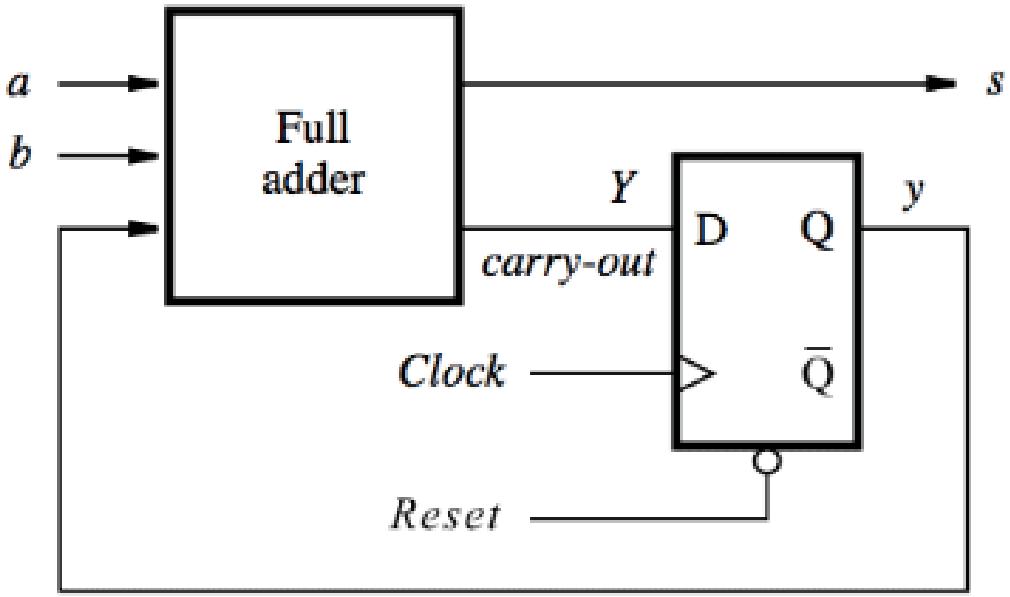


Fig: Circuit for Mealy type serial adder FSM

Serial adder - Mealy

```
module serial_add(a, b, reset, clk, sum, nst);
input a, b, clk, reset;
output reg sum;
output reg nst; /// carry out
reg cst;
parameter G = 1'b0, H = 1'b1;

initial
    cst = G; ///initially carry is 0

always @(cst,a,b)
begin
    case (cst)
    G : begin
            sum=a^b;
            if(a&b)
                nst = H;
            else
                nst = cst;
        end
    H : begin
            sum=~(a^b);
            if(~a&~b)
                nst = G;
            else
                nst = cst;
        end
    default:   nst = G;
    endcase
end // end of first always block

always@(posedge clk)
begin
    if (reset)
        cst <= G;
    else
        cst <= nst;
end

endmodule
```

Serial adder - Moore

In a Moore type FSM, output depends only on the present state.

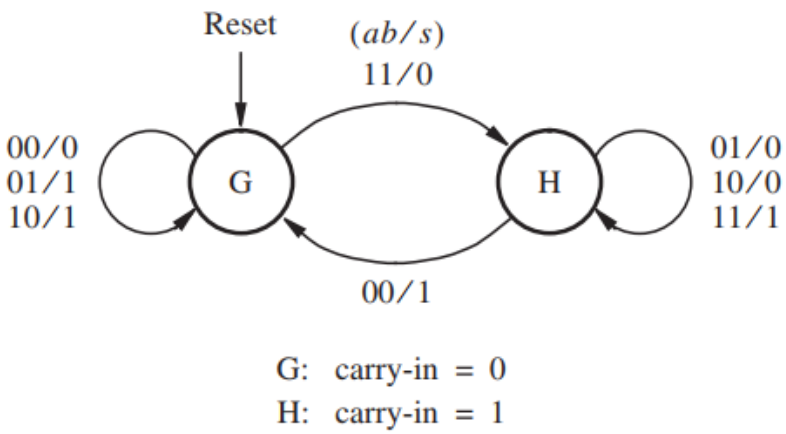
Since in both states, G and H, it is possible to produce two different outputs depending on the valuations of the inputs a and b, a Moore type FSM will need more than two states.

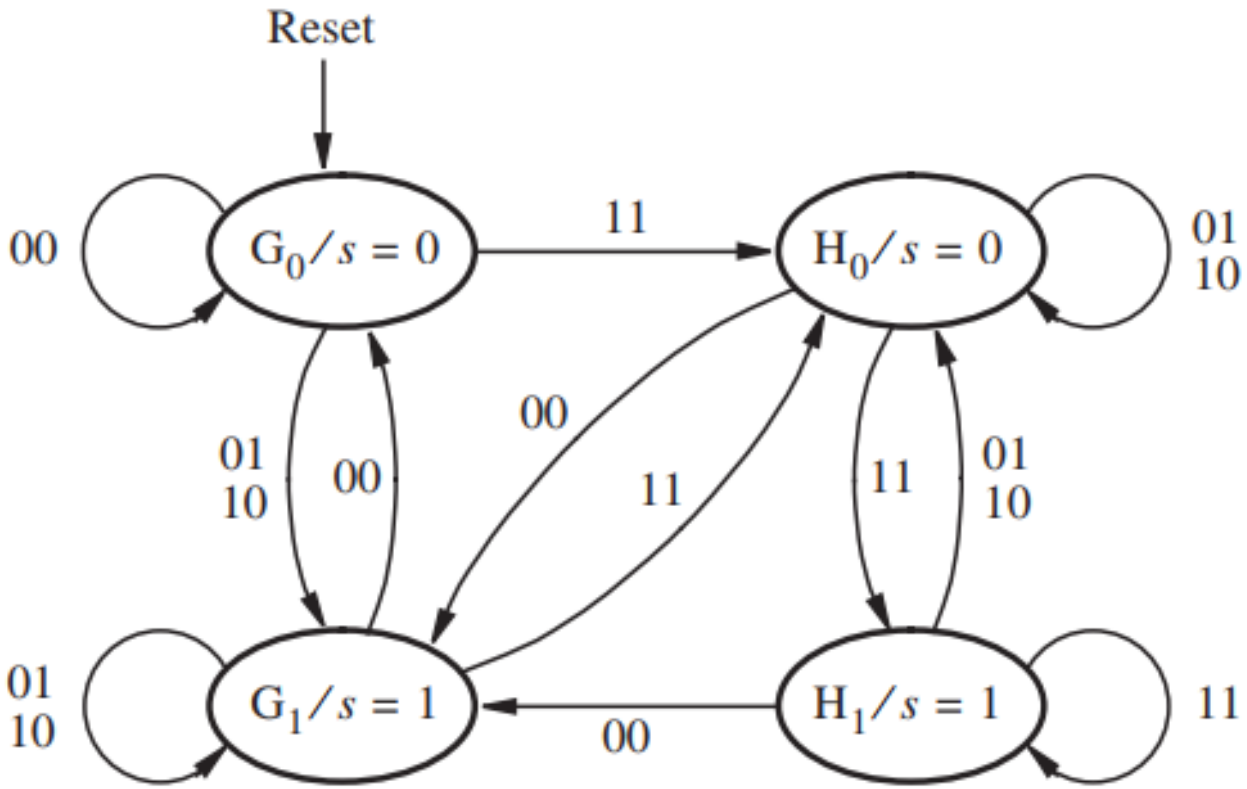
Therefore we will four states namely: G0, G1, H0 and H1.

Instead of G, we will use G0 and G1 to denote the fact that the carry is 0 and that the sum is either 0 or 1, respectively.

Similarly, instead of H, we will use H0 and H1.

Serial adder - Moore





Serial adder - Moore

Present state	Next state				Output
	$ab = 00$	01	10	11	s
G_0	G_0	G_1	G_1	H_0	0
G_1	G_0	G_1	G_1	H_0	1
H_0	G_1	H_0	H_0	H_1	0
H_1	G_1	H_0	H_0	H_1	1

Present state y_2y_1	Next state				Output
	$ab = 00$	01	10	11	s
	Y_2Y_1				
0 0	0 0	0 1	0 1	1 0	0
0 1	0 0	0 1	0 1	1 0	1
1 0	0 1	1 0	1 0	1 1	0
1 1	0 1	1 0	1 0	1 1	1

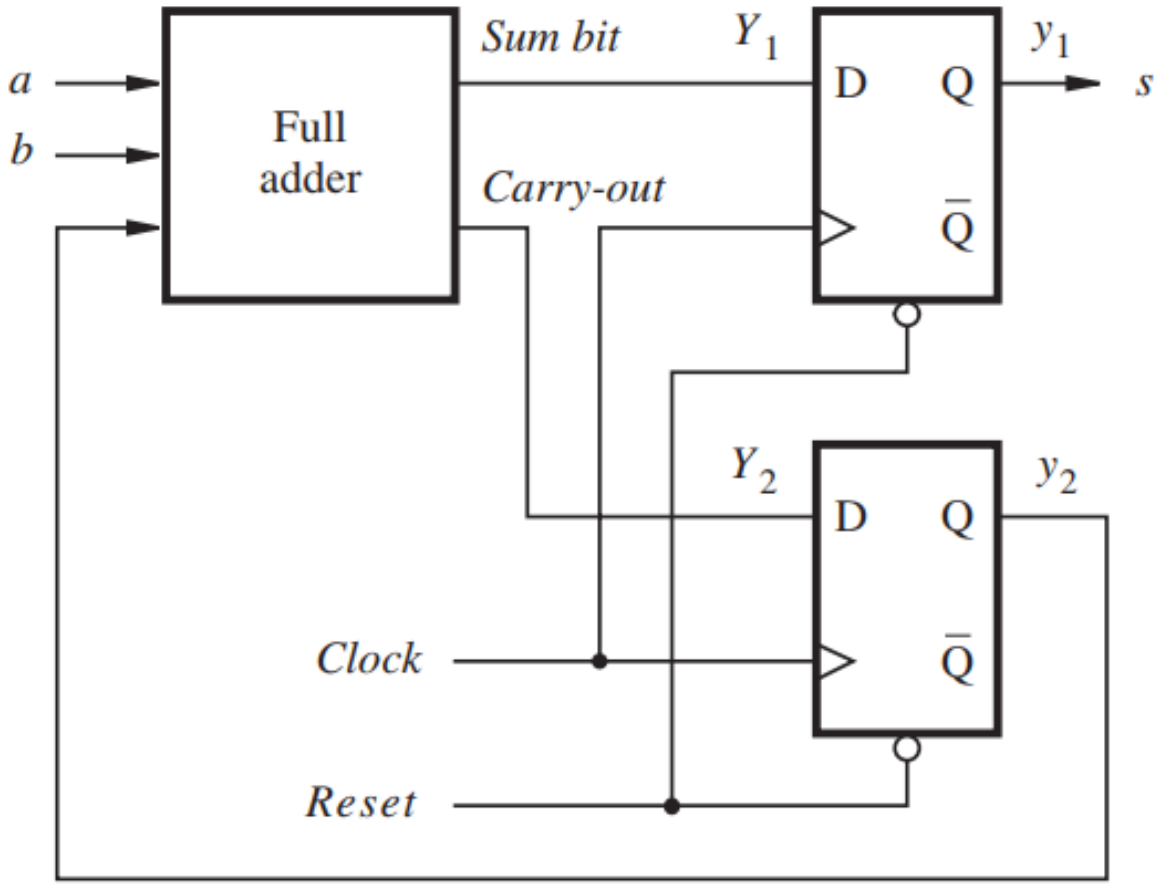
Serial adder - Moore

Present state y_2y_1	Next state $ab = 00$	01	10	11	Output s
	Y_2Y_1				
0 0	0 0	0 1	0 1	1 0	0
0 1	0 0	0 1	0 1	1 0	1
1 0	0 1	1 0	1 0	1 1	0
1 1	0 1	1 0	1 0	1 1	1

$$Y_1 = a \oplus b \oplus y_2$$

$$Y_2 = ab + ay_2 + by_2$$

$$s = y_1$$



Serial adder – Moore :Verilog Code

```
module serial_add(a, b, reset, clk, sum, c_out);
input a, b, clk, reset;
output reg sum;
output reg c_out; /// carry out
reg  [1:0] cst, nst;
parameter G0 = 2'b00,  G1 = 2'b01,  H = 2'b10,  H1=
2'b11;

initial  cst[1] = G0;

always @(cst,a,b)
begin
    case (cst)
    G0 : begin
            sum=1'b0;
            if(a&b)
                nst = H0;
            else if (a^b)
                nst = G1;
            else
                nst = G0;
        end
```

```
G1 : begin
            sum=1'b1;
             if(a&b)
                        nst = H0;
             else if (a^b)
                        nst = G1;
             else
                        nst = G0;
      end
H0 : begin
            sum=1'b0;
             if(a&b)
                        nst = H1;
             else if (a^b)
                        nst = H0;
             else
                        nst = G1;
    end
```

Serial adder – Moore : Verilog Code

```
    H1 : begin
            sum=1'b1;
             if(~a&~b)
                    nst = G1;
             else if (a^b)
                    nst = H0;
             else
                    nst = H1;
        end

    default:   nst = G;
    endcase
end // end of first always block
```

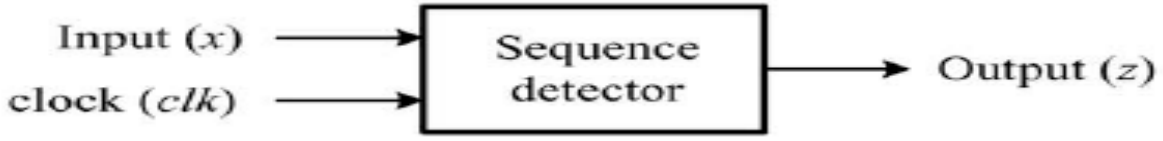
```
always@(posedge clk)
begin
    if (reset)
        cst <= G0;
    else
        cst <= nst;
end

assign c_out = cst[1];
endmodule
```

Serial adder

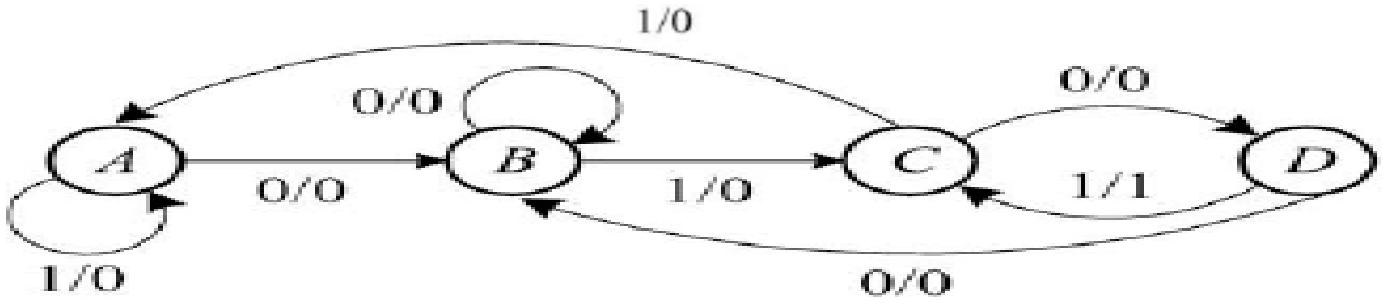
The only difference between circuits of Mealy and Moore type FSM for serial adder is that in Moore type FSM circuit, output signal ***s*** is passed through an extra flip-flop and thus delayed by one clock cycle with respect to the Mealy type FSM circuit.

A 0101 Sequence Detector *(overlapping)*

Design an FSM to detect the pattern 0101 in the input sequence *x*

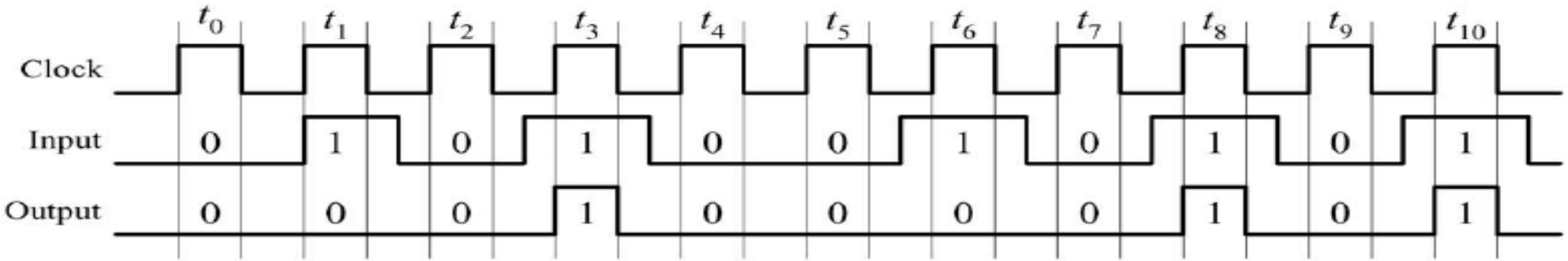


(a) Block diagram

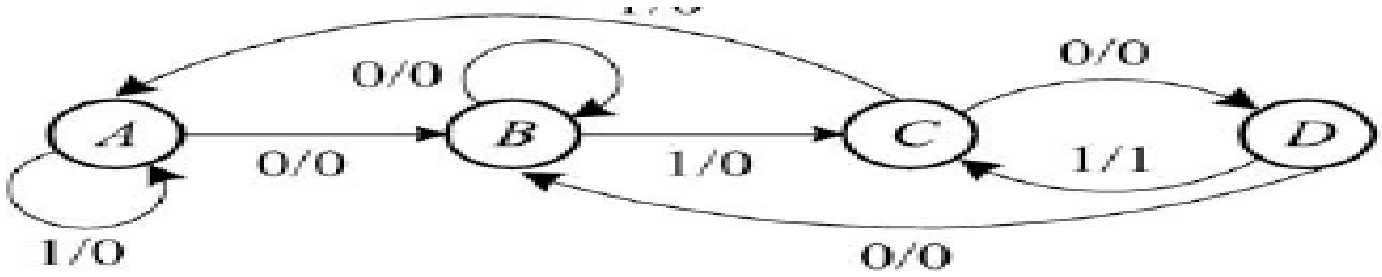


(b) State diagram

A 0101 Sequence Detector (Cont.)



(c) Timing chart



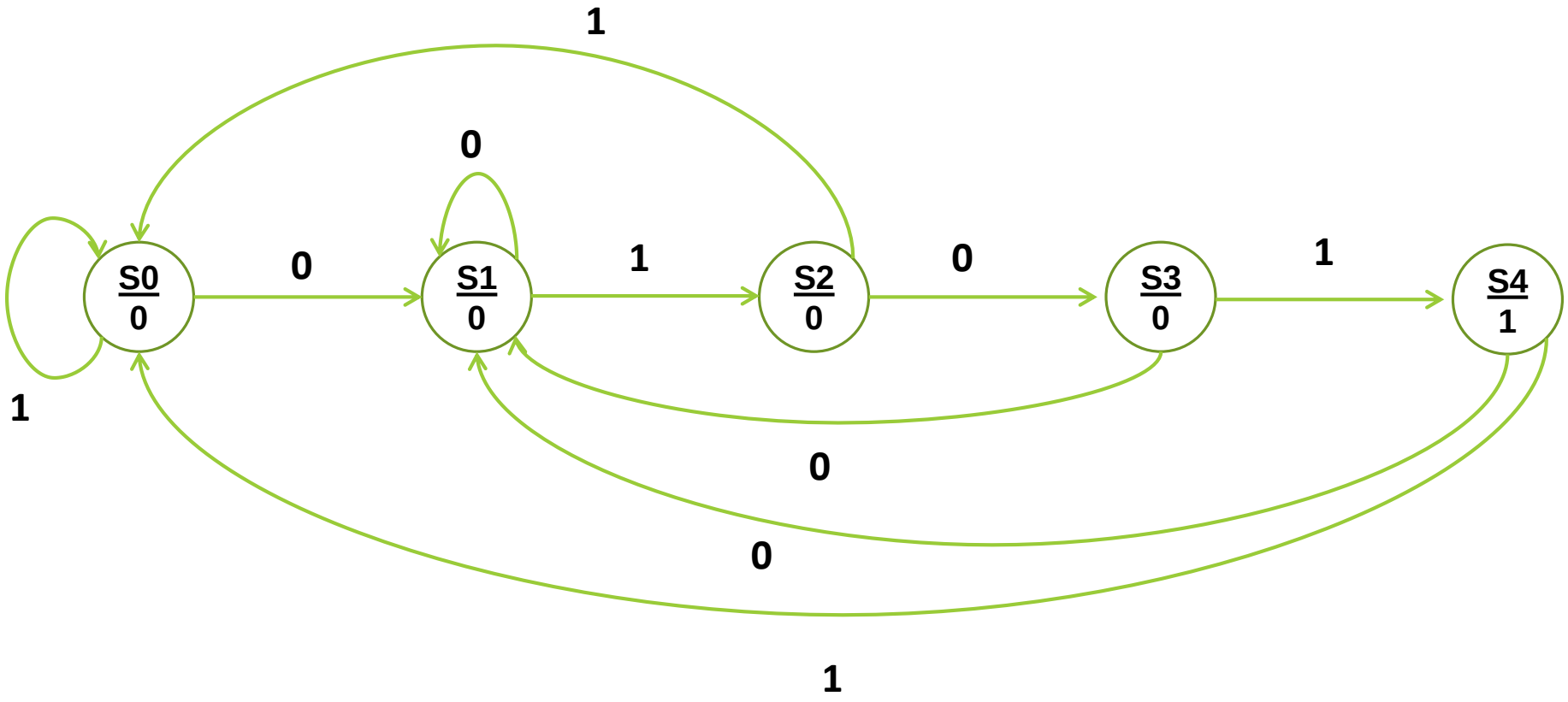
(b) State diagram

A 0101 Sequence Detector - Mealy

```
module melay_fsm(din, reset, clk, y);
input din; input clk;
 input reset;
output reg y;
reg [1:0] cst, nst;
parameter S0 = 2'b00, S1 = 2'b01, S2 =
2'b10, S3 = 2'b11; //all state
always @(cst or din)
begin
case (cst)
S0: if (din == 1'b1)
          begin nst = S0; y=1'b0; end
     else
          begin nst = S1; y=1'b0; end
S1: if (din == 1'b0)
          begin nst = S1; y=1'b0; end
     else
          begin nst = S2; y=1'b0;  end
```

```
S2: if (din == 1'b1)
     begin nst = S0; y=1'b0; end
       else
     begin nst = S3; y=1'b0; end
S3: if (din == 1'b0)
     begin nst = S1; y=1'b0; end
      else
     begin nst = S2; y=1'b1; end
default: nst = S0;
endcase
end
always@(posedge clk)
begin
if (reset) cst <= S0;
else cst <= nst;
end
endmodule
```

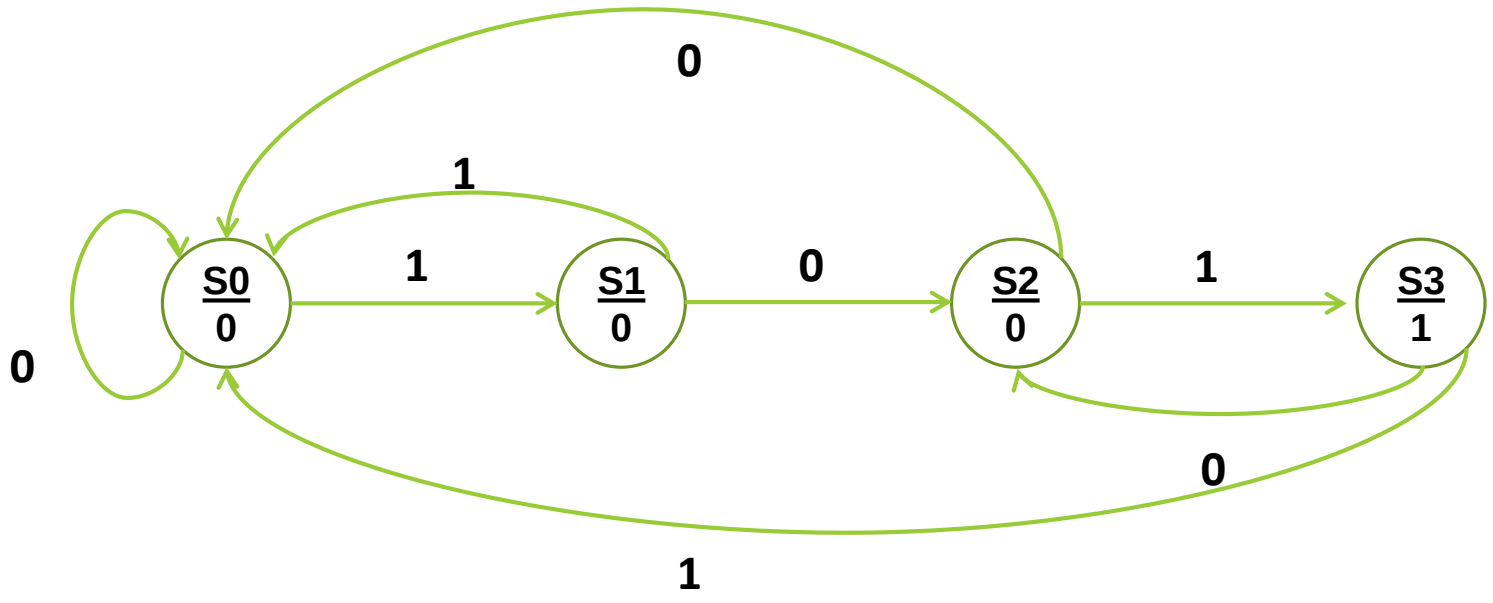
State diagram – Moore (0101)
(non-overlapping)



A 0101 Sequence Detector -Moore

```
module moore_fsm(din, reset, clk, y);
input din; input clk;
 input reset;
output reg y;
reg [2:0] cst, nst;
parameter S0 = 3'b000, S1 = 3'b001,
S2 = 3'b010, S3 = 3'b011, S4=3'b100;
//all state
always @(cst or din)
begin
case (cst)
S0: begin y=1'b0 ;
        if (din == 1'b1)  nst = S0;
        else nst = S1; end
S1: begin y=1'b0;
        if (din == 1'b0)  nst = S1;
        else nst = S2; end
S2: begin y=1'b0;
        if (din == 1'b1) nst = S0;
    else nst = S3; end
S3: begin y=1'b0;
    if (din == 1'b0)  nst = S1;
            else nst = S4; end
S4: begin y=1'b1;
    if (din == 1'b0)  nst = S1;
            else nst = S0; end
default: nst = S0;
endcase
end
always@(posedge clk)
  begin
    if (reset) cst <= S0;
    else cst <= nst;
end
endmodule
```

State diagram – Moore (0101)
(overlapping)



A 0101 Sequence Detector -Moore

```
module moore_fsm(din, reset, clk, y);
input din; input clk;
 input reset;
output reg y;
reg [1:0] cst, nst;
parameter S0 = 2'b00, S1 = 2'b01,
S2 = 2'b10, S3 = 2'b11;
//all state
always @(cst or din)
begin
case (cst)
S0: begin y=1'b0 ;
        if (din == 1'b1)  nst = S1;
        else nst = S0; end
S1: begin y=1'b0;
        if (din == 1'b0)  nst = S2;
        else nst = S0; end
S2: begin y=1'b0;
        if (din == 1'b1) nst = S3;
    else nst = S0; end
S3: begin y=1'b1;
    if (din == 1'b0)  nst = S2;
            else nst = S0; end

default: nst = S0;
endcase
end
always@(posedge clk)
begin
if (reset) cst <= S0;
else cst <= nst;
end
endmodule
```